

MyMasjidApp

Complete System Manual

 Generated: January 12, 2026

Table of Contents

1. Introduction / Overview
2. Software Development Lifecycle (SDLC)
3. Coding Standards and Guidelines
4. Version Control Guidelines
5. Software Architecture and Design Principles
6. Testing and Quality Assurance
7. Documentation Standards
8. Deployment / Release Management
9. Security Guidelines
10. Maintenance and Support
11. Tools and Environment
12. Roles and Responsibilities
13. Compliance and Legal Considerations
14. Appendices
15. Current System Information

MyMasjidApp

Complete System Manual

Project Overview

MyMasjidApp - Masjid Management System

A comprehensive full-stack application for managing masjid/madrasah operations including student management, attendance tracking, fee management, exam results, and more.

Quick Start

```
git clone <repo-url>
cd MyMasjidApp
./setup-env.sh # or setup-env.bat on Windows
./deploy.sh    # or deploy.bat on Windows
```

For Development:

For Production Deployment: See [QUICK_START.md](./QUICK_START.md) for step-by-step instructions.

Documentation

For End Users:

- [USER_GUIDE.md](./USER_GUIDE.md) -  Complete user guide for all roles (Pelajar, Guru, Pentadbir, IB)

Help Center - Access in-app help center at /help route

For Developers:

- [QUICK_START.md](./QUICK_START.md) - Get started in 5 minutes
- [DEPLOYMENT_README.md](./DEPLOYMENT_README.md) - Complete deployment guide
- [DEPLOYMENT_CHECKLIST.md](./DEPLOYMENT_CHECKLIST.md) - Deployment checklist
- [SECURITY_GUIDE.md](./SECURITY_GUIDE.md) - Security best practices
- [DEPLOYMENT_GUIDE.md](./DEPLOYMENT_GUIDE.md) - Original deployment guide

Tech Stack

Frontend:

React 19

Vite

TailwindCSS

React Router

Axios

Backend:

Node.js

Express.js

MySQL 8.0

JWT Authentication

Bcrypt for password hashing

DevOps:

Docker & Docker Compose

Nginx reverse proxy

SSL/TLS with Let's Encrypt

Features

Student Management

Teacher Management

Class Management

Attendance Tracking

Exam Management

Results Management

Fee Management

User Authentication & Authorization

Yearly Database System

Project Structure

MyMasjidApp/

```
|— backend/      # Express.js API
|  |— config/    # Configuration files
|  |— controllers/ # Request handlers
|  |— middleware/ # Auth middleware
```

```

├── routes/      # API routes
├── scripts/     # Database scripts
├── server.js    # Entry point
├── src/         # React frontend
├── database/    # SQL schema files
├── nginx/       # Nginx configuration
├── scripts/     # Deployment scripts
└── docker-compose.yml # Docker orchestration

```

Currently, two official plugins are available:

- `[@vitejs/plugin-react]`(<https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react>) uses `[Babel]`(<https://babeljs.io/>) for Fast Refresh
- `[@vitejs/plugin-react-swc]`(<https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react-swc>) uses `[SWC]`(<https://swc.rs/>) for Fast Refresh

React Compiler

The React Compiler is not enabled on this template. To add it, see [\[this documentation\]](https://react.dev/learn/react-compiler/installation)(<https://react.dev/learn/react-compiler/installation>).

Expanding the ESLint configuration

If you are developing a production application, we recommend using TypeScript with type-aware lint rules enabled. Check out the [\[TS template\]](https://github.com/vitejs/vite/tree/main/packages/create-vite/template-react-ts)(<https://github.com/vitejs/vite/tree/main/packages/create-vite/template-react-ts>) for information on how to integrate TypeScript and [\[typescript-eslint\]](https://typescript-eslint.io)(<https://typescript-eslint.io>) in your project.

Yearly Database System

Each year uses its own database:

masjid_app_2024 masjid_app_2025

Master database: masjid_master Tracks all active years and controls which DB is currently in use.

Workflow:

Admin creates a new year's database.

Selected data (students, teachers, classes) is copied over.

Old transactional data (fees, attendance, results) is reset.

New year is set active.

Year | Database | Status -----|-----|----- 2024 | masjid_app_2024 | Archived 2025 | masjid_app_2025 | Active

Benefits:

Clean yearly separation

Easy backup & restore

Preserves historical data

Improves performance

Masjid_App

User Guide

Panduan ini akan membantu anda mempelajari cara menggunakan Sistem Pengurusan Masjid dengan mudah dan efisien.

- [Memulakan](#memulakan)
- [Log Masuk](#log-masuk)
- [Panduan Mengikut Peranan](#panduan-mengikut-peranan)
- [Ciri-Ciri Utama](#ciri-ciri-utama)
- [Soalan Lazim (FAQ)](#soalan-lazim-faq)
- [Bantuan dan Sokongan](#bantuan-dan-sokongan)
-

Buka pelayar web (Chrome, Firefox, Edge, Safari)

Masukkan alamat URL sistem

Sistem akan memaparkan halaman log masuk

Masukkan Nombor IC anda

Masukkan Kata laluan anda

Klik butang "Log Masuk"

Jika anda mempunyai banyak peranan, pilih peranan yang ingin digunakan

Setelah log masuk, anda akan melihat Dashboard yang menampilkan:

Carta statistik - Maklumat ringkas tentang sistem

Aktiviti terkini - Tindakan yang baru dilakukan

Pengumuman - Notis penting dari pentadbir

Pautan pantas - Akses cepat ke fungsi utama

-

Klik "Lupa Kata Laluan?" di halaman log masuk

Pilih kaedah reset:

Email - Kod reset akan dihantar ke email anda

SMS - Kod reset akan dihantar ke telefon anda

Masukkan kod reset yang diterima

Tetapkan kata laluan baru

Klik "Daftar" di halaman log masuk

Isi borang pendaftaran dengan lengkap

Tunggu kelulusan dari pentadbir

Anda akan mendapat notifikasi setelah akaun diluluskan

-

1. Dashboard

Lihat statistik kehadiran bulanan

Cek status yuran

Lihat keputusan peperiksaan terkini

Baca pengumuman penting

2. Kehadiran

Lihat rekod kehadiran harian

Semak status kehadiran (Hadir, Tidak Hadir, Lewat, Cuti)

Lihat sejarah kehadiran mengikut bulan/tahun

Lihat peratusan kehadiran

3. Keputusan

Lihat semua keputusan peperiksaan

Semak markah dan gred

Muat turun slip keputusan

Lihat perbandingan dengan pelajar lain

4. Yuran

Lihat senarai yuran bulanan

Semak status pembayaran

Lihat baki tertunggak

Bayar yuran secara dalam talian

5. Akaun Saya

Kemas kini maklumat peribadi

Tukar kata laluan

Lihat sejarah aktiviti

✓ Log masuk setiap hari untuk melihat pengumuman terkini

✓ Semak yuran bulanan sebelum tarikh akhir

✓ Muat turun slip keputusan untuk simpanan

✓ Pastikan maklumat peribadi sentiasa dikemaskini

•

1. Dashboard

Lihat statistik kelas yang diajar

Cek pelajar yang memerlukan perhatian

Lihat jadual kelas

Baca pengumuman dari pentadbir

2. Kehadiran

Tandakan kehadiran pelajar harian

Muat naik bukti kehadiran (gambar)

Kemas kini status kehadiran (Hadir, Tidak Hadir, Lewat, Cuti)

Lihat statistik kehadiran kelas

3. Keputusan

Masukkan keputusan peperiksaan

Kemas kini markah pelajar

Muat naik slip keputusan

Lihat statistik prestasi kelas

4. Pelajar

Lihat senarai pelajar dalam kelas

Lihat profil pelajar

Semak rekod yuran pelajar

Lihat sejarah kehadiran pelajar

5. Kelas

Lihat maklumat kelas yang diajar

Lihat jadual kelas

Semak bilangan pelajar dalam kelas

- ✓ Tandakan kehadiran setiap hari selepas kelas
- ✓ Masukkan keputusan peperiksaan secepat mungkin
- ✓ Muat naik bukti kehadiran untuk rekod yang lebih baik
- ✓ Pantau pelajar yang kerap tidak hadir

•

1. Dashboard

Lihat statistik keseluruhan sistem

Pantau aktiviti terkini

Cek status sistem

Lihat notifikasi penting

2. Pengurusan Pelajar

Daftar pelajar baru

Kemas kini maklumat pelajar

Padam rekod pelajar (dengan kebenaran)

Import pelajar dari CSV

Lihat statistik pelajar

3. Pengurusan Guru

Daftar guru baru

Tetapkan guru kepada kelas

Kemas kini maklumat guru

Lihat statistik guru

4. Pengurusan Kelas

Cipta kelas baru

Tetapkan jadual kelas

Tetapkan yuran kelas

Tetapkan guru kepada kelas

Lihat statistik kelas

5. Kehadiran

Lihat semua rekod kehadiran

Tapis kehadiran mengikut kelas/tarikh

Lihat statistik kehadiran keseluruhan

Export data kehadiran

6. Yuran

Lihat semua rekod yuran

Tandakan yuran sebagai terbayar

Lihat laporan yuran

Export data yuran

7. Peperiksaan

Cipta peperiksaan baru

Tetapkan tarikh peperiksaan

Tetapkan kelas yang terlibat

Lihat senarai peperiksaan

8. Keputusan

Lihat semua keputusan peperiksaan

Tapis mengikut peperiksaan/kelas

Lihat statistik prestasi

Export data keputusan

9. Pengumuman

Cipta pengumuman baru

Tetapkan tarikh luput

Tetapkan sasaran (semua, pelajar, guru)

Kelola pengumuman aktif

10. Pengurusan Pengguna

Kelulusan pendaftaran baru

Tetapkan peranan pengguna

Reset kata laluan

Lihat Recycle Bin

11. Tetapan Sistem

Konfigurasi sistem

Tetapan gred

Tetapan pembayaran

Backup dan arkib

- ✓ Kelulusan pendaftaran baru setiap hari
- ✓ Semak statistik sistem secara berkala
- ✓ Buat backup data secara berkala
- ✓ Pantau aktiviti admin melalui "Recycle Bin"
- ✓ Pastikan pengumuman penting sentiasa terkini

-

1. Dashboard IB

Lihat laporan bulanan

Cek pembayaran yang perlu disahkan

Lihat statistik keseluruhan

2. Laporan

Lihat laporan bulanan

Semak dokumen kelas

Lihat statistik kehadiran

Lihat statistik yuran

3. Pengesahan Pembayaran

Sahkan pembayaran yuran

Lihat bukti pembayaran

Tandakan pembayaran sebagai disahkan

Lihat sejarah pengesahan

4. Pengesahan Dokumen

Sahkan dokumen kehadiran kelas

Lihat gambar bukti kehadiran

Tandakan dokumen sebagai disahkan

- ✓ Semak laporan bulanan setiap bulan
- ✓ Sahkan pembayaran dengan teliti
- ✓ Semak bukti pembayaran sebelum mengesahkan
- ✓ Pastikan semua dokumen telah disahkan sebelum akhir bulan

•

Dashboard adalah halaman utama sistem yang menampilkan:

Statistik ringkas - Nombor penting tentang sistem

Carta visual - Grafik untuk analisis data

Aktiviti terkini - Sejarah tindakan terkini

Pengumuman - Notis penting dari pentadbir

Pautan pantas - Akses cepat ke fungsi utama

Semasa mengisi borang pendaftaran:

Isi semua medan yang bertanda () wajib

Nombor IC mesti dalam format: XXXXXX-XX-XXXX

Nombor telefon mesti dalam format: 01X-XXXXXXX

Pastikan email adalah valid (jika diisi)

Semak semua maklumat sebelum hantar

Di kebanyakan halaman, anda boleh:

Cari - Gunakan kotak carian untuk mencari rekod

Tapis - Gunakan penapis untuk menyempitkan hasil

Urutkan - Klik pada tajuk lajur untuk mengurutkan

Halaman - Gunakan navigasi halaman untuk melihat lebih banyak

Sistem ini boleh digunakan pada:

 Komputer desktop

 Telefon pintar

 Tablet

Sesuaikan saiz skrin secara automatik untuk pengalaman terbaik.

Sistem akan memaparkan notifikasi untuk:

 Tindakan berjaya

 Tindakan gagal

 Amaran penting

 Maklumat

Notifikasi akan hilang secara automatik selepas beberapa saat.

•

A:

Log masuk ke sistem

Pergi ke "Akaun Saya" atau "Tetapan"

Klik "Tukar Kata Laluan"

Masukkan kata laluan lama dan kata laluan baru

Klik "Simpan"

A: Gunakan penapis tarikh di halaman yang berkaitan untuk melihat rekod pada tarikh tertentu.

A: Ya, anda boleh memuat naik:

Gambar bukti kehadiran (format: JPG, PNG, maksimum 5MB)

Slip keputusan (format: JPG, PNG, PDF, maksimum 10MB)

Bukti pembayaran (format: JPG, PNG, maksimum 5MB)

A:

Pergi ke halaman yang ingin diexport

Gunakan penapis untuk menyempitkan data

Klik butang "Export" atau "Muat Turun"

Pilih format (Excel, PDF, CSV)

Fail akan dimuat turun

A: Sesi anda akan tamat secara automatik selepas 24 jam. Untuk keselamatan, adalah disyorkan untuk log keluar selepas setiap penggunaan.

A: Ya, anda boleh log masuk dari mana-mana peranti dengan menggunakan nombor IC dan kata laluan anda. Namun, untuk keselamatan, sistem akan log keluar peranti lain jika anda log masuk dari peranti baharu.

A:

Pergi ke halaman "Hubungi Kami"

Isi borang bantuan dengan lengkap

Hantar permintaan

Anda akan mendapat respons dalam masa 1-2 hari bekerja

A:

Aktif - Rekod aktif dan boleh digunakan

Tidak Aktif - Rekod tidak aktif (tidak boleh digunakan)

Tertunggak - Yuran belum dibayar

Terbayar - Yuran telah dibayar

Hadir - Pelajar hadir ke kelas

Tidak Hadir - Pelajar tidak hadir ke kelas

Lewat - Pelajar lewat ke kelas

Cuti - Pelajar bercuti

A:

Pelajar/Guru: Pergi ke "Akaun Saya" untuk melihat sejarah aktiviti

Pentadbir: Pergi ke "Recycle Bin" untuk melihat semua tindakan admin

-

Jika anda menghadapi sebarang masalah atau soalan:

Pergi ke halaman "Hubungi Kami"

Isi borang dengan lengkap

Terangkan masalah anda dengan jelas

Lampirkan tangkapan skrin jika perlu

Hantar permintaan

Pengumuman - Semak pengumuman untuk maklumat terkini

Dokumentasi - Baca dokumentasi lengkap di halaman Bantuan

Video Tutorial - Tonton video tutorial (jika ada)

Jika anda menemui bug atau ralat dalam sistem:

Pergi ke "Hubungi Kami"

Pilih kategori "Laporan Bug"

Terangkan bug dengan jelas

Lampirkan tangkapan skrin

Terangkan langkah-langkah untuk menghasilkan bug

-

Gunakan Ctrl+F (Windows) atau Cmd+F (Mac) untuk mencari di halaman

Klik pada ikon (?) untuk mendapatkan bantuan konteks

Gunakan butang Undo jika tersilap melakukan tindakan (dalam masa 1 jam)

Semak pengumuman setiap hari untuk maklumat terkini

Buat backup data setiap minggu

Arsipkan data tahun lalu untuk prestasi yang lebih baik

Gunakan export untuk menyimpan data penting

Pantau aktiviti admin untuk audit

Tandakan kehadiran sejurus selepas kelas

Masukkan keputusan secepat mungkin selepas peperiksaan

Gunakan penapis untuk mencari pelajar dengan cepat
Semak notifikasi untuk pelajar yang memerlukan perhatian
Log masuk setiap hari untuk melihat pengumuman
Semak yuran bulanan sebelum tarikh akhir
Muat turun slip keputusan untuk simpanan
Kemas kini maklumat peribadi jika ada perubahan

•

IC - Nombor Kad Pengenalan
IB - Imam Bilal (Pengesah Pembayaran)
PIC - Person In Charge (Orang Bertanggungjawab)
Yuran - Bayaran bulanan kelas
Tunggak - Bayaran yang belum dibayar
Terbayar - Bayaran yang telah dibayar
Kehadiran - Rekod hadir ke kelas
Keputusan - Markah dan gred peperiksaan
Dashboard - Halaman utama sistem
Pengumuman - Notis penting dari pentadbir

•

Panduan ini dikemaskini pada: Disember 2025

- ✓ Menambah panduan untuk peranan IB
- ✓ Menambah bahagian Tips dan Trik
- ✓ Menambah lebih banyak FAQ
- ✓ Menambah glosari

•

Sila simpan panduan ini untuk rujukan masa hadapan!

Jika anda mempunyai sebarang cadangan untuk meningkatkan panduan ini, sila hubungi kami melalui halaman "Hubungi Kami".

System Workflows

This document provides comprehensive workflow documentation with Mermaid diagrams for all major system workflows in MyMasjidApp.

- [Authentication Workflows](#1-authentication-workflows)

User Registration (Student self-registration)

Login and Session Management

Password Recovery (Email & SMS methods)

- [Student Management Workflows](#2-student-management-workflows)

Student Registration (Admin-created)

Student Enrollment to Classes

Student Profile Management

- [Class Management Workflows](#3-class-management-workflows)

Class Creation

Class Assignment (Students to Classes)

Teacher Assignment to Classes

- [Attendance Workflows](#4-attendance-workflows)

Marking Attendance (Individual & Bulk)

Attendance Tracking & History

Attendance Approval (with proof/document)

- [Fee Management Workflows](#5-fee-management-workflows)

Fee Generation (Monthly)

Fee Payment Process

PIC Approval Workflow

Fee Document Confirmation

- [Exam & Results Workflows](#6-exam--results-workflows)

Exam Creation

Result Entry & Management

Grade Calculation

- [Role-Specific Workflows](#7-role-specific-workflows)

Admin Workflows

Teacher Workflows

Student Workflows

IB (Payment Approver) Workflows

- [System-Level Workflows](#8-system-level-workflows)

Yearly Database Migration

Admin Actions & Snapshots

-

flowchart TD

```
Start([User visits Registration Page]) --> ValidateIC[Validate IC Format<br/>12 digits]
ValidateIC -->|Invalid| ShowError[Show Error:<br/>Format IC tidak sah]
ShowError --> Start
ValidateIC -->|Valid| NormalizeIC[Normalize IC<br/>Remove hyphens/spaces]
NormalizeIC --> CheckDuplicate[Check for<br/>Duplicate IC]
CheckDuplicate -->|Exists| ShowDuplicateError[Show Error:<br/>IC already registered]
ShowDuplicateError --> Start
CheckDuplicate -->|Not Exists| ValidateEmail[Email<br/>provided?]
ValidateEmail -->|Yes| CheckEmailDuplicate[Email<br/>already exists?]
CheckEmailDuplicate -->|Yes| ShowEmailError[Show Error:<br/>Email already registered]
ShowEmailError --> Start
CheckEmailDuplicate -->|No| ValidatePassword[Validate Password<br/>Min 6 characters]
ValidateEmail -->|No| ValidatePassword
ValidatePassword -->|Invalid| ShowPasswordError[Show Error:<br/>Password too weak]
ShowPasswordError --> Start
ValidatePassword -->|Valid| HashPassword[Hash Password<br/>bcrypt, 12 rounds]
HashPassword --> CreateUser[Create User Record<br/>role='student']
CreateUser --> CreateStudent[Create Student Record<br/>in students table]
CreateStudent --> Success[Registration Success<br/>Redirect to Login]
Success --> End([End])
```

Key Steps:

User enters IC number, name, email (optional), password (optional), phone, age

System validates IC format (12 digits)

System normalizes IC (removes hyphens/spaces)

System checks for duplicate IC/email

If password provided, system validates strength (min 6 chars)

System hashes password with bcrypt (12 rounds)

System creates user record with role='student'

System creates student record in students table

User is redirected to login page

API Endpoint: POST /auth/register

Database Tables:

users - Main user record

students - Student-specific data

Error Handling:

Invalid IC format → "Format IC tidak sah. Sila masukkan 12 digit nombor IC."

Duplicate IC → "IC number already registered"

Duplicate email → "Emel ini sudah didaftarkan. Sila gunakan emel lain atau log masuk."

Weak password → Password strength validation message

-

flowchart TD

```
graph TD
    Start([User visits Login Page]) --> ValidateIC[Validate IC Format]
    ValidateIC -->|Invalid| ShowError[Show Error:<br/>Format IC tidak sah]
    ShowError --> Start
    ValidateIC -->|Valid| CheckLocked[Account<br/>Locked?]
    CheckLocked -->|Yes| ShowLockedError[Show Error:<br/>Account locked<br/>Try again after X minutes]
    ShowLockedError --> End([End])
    CheckLocked -->|No| NormalizeIC[Normalize IC]
    NormalizeIC --> FindUser[Find User<br/>by IC]
    FindUser -->|Not Found| RecordFailedAttempt[Record Failed Attempt]
```

```

RecordFailedAttempt --> CheckMaxAttempts{Max Attempts<br/>Reached?}
CheckMaxAttempts -->|Yes| LockAccount[Lock Account<br/>15 minutes]
LockAccount --> ShowLockedError
CheckMaxAttempts -->|No| ShowLoginError[Show Error:<br/>Invalid credentials]
ShowLoginError --> Start
FindUser -->|Found| VerifyPassword{Verify Password<br/>bcrypt compare}
VerifyPassword -->|Invalid| RecordFailedAttempt
VerifyPassword -->|Valid| CheckStatus{Check Account<br/>Status}
CheckStatus -->|pending| ShowPendingError[Show Error:<br/>Account pending approval]
ShowPendingError --> End
CheckStatus -->|tidak_aktif| ShowInactiveError[Show Error:<br/>Account inactive]
ShowInactiveError --> End
CheckStatus -->|aktif| FetchRoles[Fetch User Roles<br/>from user_roles table]
FetchRoles --> DetermineActiveRole{Requested Role<br/>Available?}
DetermineActiveRole -->|Yes| SetActiveRole[Set Active Role]
DetermineActiveRole -->|No| UsePrimaryRole[Use Primary Role]
SetActiveRole --> GenerateTokens[Generate JWT Tokens<br/>Access + Refresh]
UsePrimaryRole --> GenerateTokens
GenerateTokens --> RecordSuccess[Record Successful Login<br/>Update last_login]
RecordSuccess --> ReturnTokens[Return Tokens<br/>+ User Info + Roles]
ReturnTokens --> StoreTokens[Frontend Stores Tokens<br/>in localStorage]
StoreTokens --> Redirect[Redirect to Dashboard<br/>Based on Role]
Redirect --> End

```

Key Steps:

User enters IC number and password

System validates IC format

System checks if account is locked (too many failed attempts)

System normalizes IC and finds user

System verifies password using bcrypt

System checks account status (pending/aktif/tidak_aktif)

System fetches all user roles from user_roles table

System determines active role (requested role if available, else primary role)

System generates JWT access token (24h) and refresh token (7 days)

System records successful login and updates last_login

Frontend stores tokens and redirects to role-appropriate dashboard

API Endpoint: POST /auth/login

Database Tables:

users - User authentication data

user_roles - User role assignments

login_attempts - Failed login tracking

Session Management:

Access Token: 24 hours validity

Refresh Token: 7 days validity (if implemented)

Tokens stored in localStorage (frontend)

Security Features:

Account lockout after 5 failed attempts (15 minutes)

Password hashing with bcrypt (12 rounds)

Role-based access control (RBAC)

-

flowchart TD

```
graph TD
    Start([User visits Forgot Password]) --> EnterIC[User Enters IC Number]
    EnterIC --> ValidateIC[Validate IC Format]
    ValidateIC --> Invalid[Invalid]
    Invalid --> ShowError[Show Error]
    ShowError --> Start
    ValidateIC --> Valid[Valid]
    Valid --> NavigateToMethod[Choose Reset Method]
    NavigateToMethod --> CheckOptions[Check Reset Options<br/>POST /auth/check-reset-options]
    CheckOptions --> HasEmail{User has<br/>Email?}
    HasEmail --> YesEmail[Yes]
    YesEmail --> ShowEmailOption[Show Email Option<br/>Masked email]
    HasEmail --> NoEmail[No]
    NoEmail --> HideEmailOption[Hide Email Option]
    CheckOptions --> HasPhone{User has<br/>Phone?}
    HasPhone --> YesPhone[Yes]
    YesPhone --> ShowPhoneOption[Show Phone Option<br/>Masked phone]
    HasPhone --> NoPhone[No]
    NoPhone --> HidePhoneOption[Hide Phone Option]
    ShowEmailOption --> UserChoosesEmail{User Chooses<br/>Method}
    ShowPhoneOption --> UserChoosesPhone{User Chooses<br/>Method}
    UserChoosesEmail --> Email[Email]
    Email --> GenerateEmailToken[Generate 32-byte<br/>Hex Token]
    GenerateEmailToken --> StoreToken[Store Token in DB<br/>24h expiry]
    StoreToken --> SendEmail[Send Email with<br/>Reset Link]
    SendEmail --> EmailSent[Email Sent<br/>Show Success Message]
    UserChoosesPhone --> Phone[Phone]
    Phone --> GenerateCode[Generate 6-digit<br/>Numeric Code]
```

```

GenerateCode --> StoreCode[Store Code in DB<br/>10min expiry]
StoreCode --> SendSMS[Send SMS with<br/>Reset Code]
SendSMS --> SMSSent[SMS Sent<br/>Show Success Message]
EmailSent --> WaitForUser[Wait for User Action]
SMSSent --> WaitForUser
WaitForUser -->|Email| UserClicksLink[User Clicks<br/>Reset Link]
UserClicksLink --> NavigateToReset[Navigate to<br/>/reset-password?token=...]
NavigateToReset --> EnterNewPassword[User Enters<br/>New Password]
WaitForUser -->|Phone| UserEntersCode[User Enters<br/>6-digit Code]
UserEntersCode --> NavigateToCodeReset[Navigate to<br/>/reset-password-code?ic=...]
NavigateToCodeReset --> EnterNewPassword
EnterNewPassword --> ValidatePassword{Validate Password<br/>Min 6 chars}
ValidatePassword -->|Invalid| ShowPasswordError[Show Error]
ShowPasswordError --> EnterNewPassword
ValidatePassword -->|Valid| VerifyToken{Verify Token/Code<br/>Not expired<br/>Not used}
VerifyToken -->|Invalid| ShowTokenError[Show Error:<br/>Invalid/expired token]
ShowTokenError --> End([End])
VerifyToken -->|Valid| HashNewPassword[Hash New Password<br/>bcrypt, 12 rounds]
HashNewPassword --> UpdatePassword[Update User Password]
UpdatePassword --> MarkTokenUsed[Mark Token/Code<br/>as Used]
MarkTokenUsed --> Success[Password Reset Success<br/>Redirect to Login]
Success --> End

```

Key Steps:

Email Method:

User enters IC number on forgot password page

System checks available reset methods (email/phone)

User selects email method

System generates 32-byte hexadecimal token

System stores token in password_reset_tokens table (24h expiry)

System sends email with reset link: {FRONTEND_URL}/reset-password?token={token}

User clicks link and enters new password

System validates token (not expired, not used)

System hashes new password and updates user record

System marks token as used

User redirected to login page

Phone Method:

User enters IC number on forgot password page

System checks available reset methods (email/phone)

User selects phone method

System generates 6-digit numeric code

System stores code in password_reset_tokens table (10min expiry)

System sends SMS with reset code

User navigates to /reset-password-code?ic={icNumber} and enters code

System validates code (not expired, not used)

System hashes new password and updates user record

System marks code as used

User redirected to login page

API Endpoints:

POST /auth/check-reset-options - Check available reset methods

POST /auth/request-reset-email - Request email reset

POST /auth/request-reset-phone - Request phone reset

POST /auth/reset-password - Reset password with token/code

Database Tables:

password_reset_tokens - Stores reset tokens/codes

users - User password storage

Security Features:

Email tokens: 24-hour expiry, one-time use

Phone codes: 10-minute expiry, one-time use

User enumeration prevention (generic messages)

Password strength validation (min 6 characters)

-

flowchart TD

```
Start([Admin Creates Student]) --> ValidateInput[Validate Input:<br/>IC, Name, Email, Phone, Age]
ValidateInput -->|Invalid| ShowError[Show Validation Error]
ShowError --> Start
ValidateInput -->|Valid| CheckICExists{IC Already<br/>Exists?}
CheckICExists -->|Yes| ShowICError[Show Error:<br/>IC already exists]
ShowICError --> Start
CheckICExists -->|No| CheckEmailExists{Email Already<br/>Exists?}
CheckEmailExists -->|Yes| ShowEmailError[Show Error:<br/>Email already exists]
ShowEmailError --> Start
CheckEmailExists -->|No| NormalizeIC[Normalize IC]
NormalizeIC --> CreateUser[Create User Record<br/>role='student']
CreateUser --> CreateStudent[Create Student Record<br/>kelas_id optional]
CreateStudent --> CreateSnapshot[Create Admin Action<br/>Snapshot for Undo]
CreateSnapshot --> Success[Student Created Successfully]
Success --> End([End])
```

Key Steps:

Admin enters student details (IC, name, email, phone, age, address)

System validates all input fields

System checks for duplicate IC/email

System normalizes IC format

System creates user record with role='student'

System creates student record (class assignment optional)

System creates admin action snapshot (for undo functionality)

Success response returned

API Endpoint: POST /students (Admin only)

Database Tables:

users - User record

students - Student record (links to class)

admin_action_snapshots - Undo tracking

-

flowchart TD

```
Start([Enroll Student to Class]) --> ValidateStudent{Student<br/>Exists?}
ValidateStudent -->|No| ShowError[Show Error:<br/>Student not found]
ShowError --> End([End])
ValidateStudent -->|Yes| ValidateClass{Class<br/>Exists?}
ValidateClass -->|No| ShowClassError[Show Error:<br/>Class not found]
ShowClassError --> End
ValidateClass -->|Yes| CheckCapacity{Class has<br/>Available Capacity?}
CheckCapacity -->|No| ShowCapacityError[Show Error:<br/>Class is full]
ShowCapacityError --> End
CheckCapacity -->|Yes| CheckAlreadyEnrolled{Student Already<br/>Enrolled?}
CheckAlreadyEnrolled -->|Yes| ShowEnrolledError[Show Error:<br/>Already enrolled]
ShowEnrolledError --> End
CheckAlreadyEnrolled -->|No| UpdateStudent[Update Student Record<br/>Set kelas_id]
UpdateStudent --> UpdateEnrollmentDate[Set tarikh_daftar<br/>to current date]
UpdateEnrollmentDate --> CreateSnapshot[Create Admin Action<br/>Snapshot]
CreateSnapshot --> Success[Enrollment Success]
Success --> End
```

Key Steps:

Admin selects student and class

System validates student exists

System validates class exists

System checks class capacity (kapasiti field)

System checks if student already enrolled

System updates student record with kelas_id

System sets enrollment date (tarikh_daftar)

System creates admin action snapshot

Success response returned

API Endpoint: PUT /students/:ic (Admin/Teacher)

Database Tables:

students - Student enrollment data

classes - Class capacity tracking

-

flowchart TD

```
Start([Update Student Profile]) --> Authenticate{User Authenticated?}
Authenticate -->|No| ShowAuthError[Show Error:<br/>Unauthorized]
ShowAuthError --> End([End])
Authenticate -->|Yes| CheckPermission{Has Permission?<br/>Admin/Teacher/Own Profile}
CheckPermission -->|No| ShowPermissionError[Show Error:<br/>Access denied]
ShowPermissionError --> End
CheckPermission -->|Yes| ValidateInput[Validate Input Data]
ValidateInput -->|Invalid| ShowValidationError[Show Validation Error]
ShowValidationError --> Start
ValidateInput -->|Valid| UpdateUser[Update User Record<br/>Name, Email, Phone, Age, Address]
UpdateUser --> UpdateStudent{Student-specific<br/>Fields?}
UpdateStudent -->|Yes| UpdateStudentRecord[Update Student Record<br/>kelas_id,
tarikh_daftar]
UpdateStudent -->|No| CreateSnapshot[Create Admin Action<br/>Snapshot]
UpdateStudentRecord --> CreateSnapshot
CreateSnapshot --> Success[Profile Updated Successfully]
Success --> End
```

Key Steps:

User requests to update student profile

System authenticates user

System checks permissions (admin can update any, teacher can update their students, student can update own)

System validates input data

System updates user record (name, email, phone, age, address)

If student-specific fields changed, update students table

System creates admin action snapshot (if admin action)

Success response returned

API Endpoint: PUT /students/:id (Admin/Teacher/Student for own profile)

Database Tables:

users - User profile data

students - Student-specific data

-

flowchart TD

```
Start([Admin Creates Class]) --> ValidateInput[Validate Input:<br/>Name, Level, Schedule, Yuran, Capacity]
ValidateInput -->|Invalid| ShowError[Show Validation Error]
ShowError --> Start
ValidateInput -->|Valid| ValidateTeacher{Teacher IC<br/>Provided?}
ValidateTeacher -->|Yes| CheckTeacherExists{Teacher<br/>Exists?}
CheckTeacherExists -->|No| ShowTeacherError[Show Error:<br/>Teacher not found]
ShowTeacherError --> Start
CheckTeacherExists -->|Yes| CreateClass[Create Class Record]
ValidateTeacher -->|No| CreateClass
CreateClass --> SetDefaultStatus[Set Status='aktif'<br/>Set Default Capacity=20]
SetDefaultStatus --> CreateSnapshot[Create Admin Action<br/>Snapshot]
CreateSnapshot --> Success[Class Created Successfully]
Success --> End([End])
```

Key Steps:

Admin enters class details (name, level, schedule, sessions, yuran, teacher IC, capacity)

System validates all input fields

If teacher IC provided, system validates teacher exists

System creates class record

System sets default status='aktif' and capacity=20 if not specified

System creates admin action snapshot

Success response returned

API Endpoint: POST /classes (Admin only)

Database Tables:

classes - Class information

users - Teacher reference (guru_ic)

admin_action_snapshots - Undo tracking

-

This workflow is covered in [Section 2.2 Student Enrollment to Classes](#22-student-enrollment-to-classes).

-

flowchart TD

```
Start([Assign Teacher to Class]) --> ValidateClass{Class<br/>Exists?}
ValidateClass -->|No| ShowClassError[Show Error:<br/>Class not found]
ShowClassError --> End([End])
ValidateClass -->|Yes| ValidateTeacher{Teacher<br/>Exists?}
ValidateTeacher -->|No| ShowTeacherError[Show Error:<br/>Teacher not found]
ShowTeacherError --> End
ValidateTeacher -->|Yes| CheckTeacherRole{User has<br/>teacher role?}
CheckTeacherRole -->|No| ShowRoleError[Show Error:<br/>User is not a teacher]
ShowRoleError --> End
CheckTeacherRole -->|Yes| UpdateClass[Update Class Record<br/>Set guru_ic]
UpdateClass --> CreateSnapshot[Create Admin Action<br/>Snapshot]
CreateSnapshot --> Success[Teacher Assigned Successfully]
Success --> End
```

Key Steps:

Admin selects class and teacher

System validates class exists

System validates teacher exists and has teacher role

System updates class record with guru_ic

System creates admin action snapshot

Success response returned

API Endpoint: PUT /classes/:id (Admin only)

Database Tables:

classes - Class teacher assignment

users - Teacher validation

admin_action_snapshots - Undo tracking

-

flowchart TD

```
Start([Mark Attendance]) --> CheckPermission{Has Permission?<br/>Admin/Teacher}
CheckPermission -->|No| ShowError[Show Error:<br/>Unauthorized]
ShowError --> End([End])
CheckPermission -->|Yes| ValidateInput[Validate Input:<br/>Student IC, Class ID, Date, Status]
ValidateInput -->|Invalid| ShowValidationError[Show Validation Error]
ShowValidationError --> Start
```

```
ValidateInput -->|Valid| CheckBulk{Bulk or<br/>Individual?}
CheckBulk -->|Individual| CheckExisting{Attendance<br/>Already Exists?}
CheckExisting -->|Yes| UpdateAttendance[Update Existing<br/>Attendance Record]
CheckExisting -->|No| CreateAttendance[Create New<br/>Attendance Record]
CheckBulk -->|Bulk| ProcessBulk[Process Array of<br/>Attendance Records]
ProcessBulk --> ValidateAll{All Records<br/>Valid?}
ValidateAll -->|No| ShowBulkError[Show Error:<br/>Invalid records]
ShowBulkError --> Start
ValidateAll -->|Yes| BulkInsert[Insert/Update All<br/>Attendance Records]
UpdateAttendance --> CreateSnapshot[Create Admin Action<br/>Snapshot]
CreateAttendance --> CreateSnapshot
BulkInsert --> CreateSnapshot
CreateSnapshot --> Success[Attendance Marked Successfully]
Success --> End
```

Key Steps:

Teacher/Admin selects class and date

System validates permissions (admin or teacher of the class)

For individual: System validates student IC, class ID, date, status

For bulk: System validates array of attendance records

System checks if attendance already exists for student+class+date

If exists: Update existing record

If not exists: Create new record

System creates admin action snapshot

Success response returned

API Endpoints:

POST /attendance - Mark individual attendance

POST /attendance/bulk - Mark bulk attendance

Database Tables:

attendance - Attendance records

admin_action_snapshots - Undo tracking

Attendance Status Values:

Hadir - Present

Tidak Hadir - Absent

Cuti - On Leave

-

flowchart TD

```
Start([View Attendance]) --> CheckPermission{Has Permission?}
CheckPermission -->|No| ShowError[Show Error:<br/>Unauthorized]
ShowError --> End([End])
CheckPermission -->|Yes| CheckRole{User Role?}
CheckRole -->|Admin| GetAllAttendance[Get All Attendance<br/>with Filters]
CheckRole -->|Teacher| GetClassAttendance[Get Attendance for<br/>Assigned Classes Only]
CheckRole -->|Student| GetOwnAttendance[Get Own Attendance<br/>Only]
GetAllAttendance --> ApplyFilters[Apply Filters:<br/>Date Range, Class, Student, Status]
GetClassAttendance --> ApplyFilters
GetOwnAttendance --> ApplyFilters
ApplyFilters --> Paginate[Apply Pagination]
Paginate --> ReturnData[Return Attendance Data<br/>with Pagination Info]
ReturnData --> End
```

Key Steps:

User requests attendance data

System checks user permissions

System applies role-based filtering:

Admin: All attendance records

Teacher: Only their assigned classes

Student: Only their own attendance

System applies optional filters (date range, class, student, status)

System applies pagination

System returns attendance data with pagination metadata

API Endpoint: GET /attendance (with query parameters)

Query Parameters:

start_date - Start date (YYYY-MM-DD)

end_date - End date (YYYY-MM-DD)

date - Single date or month (YYYY-MM or YYYY-MM-DD)

class_id - Filter by class

student_ic - Filter by student

status - Filter by status

page - Page number

limit - Records per page

Database Tables:

attendance - Attendance records

classes - Class information (for teacher filtering)

students - Student information

-

flowchart TD

```
Start([Mark Attendance with Proof]) --> CheckPermission{Has  
Permission?<br/>Admin/Teacher}  
CheckPermission -->|No| ShowError[Show Error]  
ShowError --> End([End])  
CheckPermission -->|Yes| UploadProof[Upload Proof Document<br/>Image/File]  
UploadProof --> ValidateFile{File Valid?}  
ValidateFile -->|No| ShowFileError[Show Error:<br/>Invalid file]  
ShowFileError --> Start  
ValidateFile -->|Yes| SaveFile[Save File to<br/>uploads/attendance/]  
SaveFile --> CreateAttendance[Create Attendance Record<br/>with proof_path]  
CreateAttendance --> StoreProofPath[Store Proof Path<br/>in Attendance Record]  
StoreProofPath --> WaitConfirmation[Wait for Admin/PIC/IB<br/>Document Confirmation]  
WaitConfirmation --> AdminReviews{Admin/PIC/IB Reviews<br/>Proof Document}  
AdminReviews -->|Confirm| ConfirmDocument[Confirm  
Document<br/>document_confirmed=true<br/>confirmed_by, confirmed_at]  
AdminReviews -->|Reject| SetRejected[Update Status or<br/>Leave as is]  
ConfirmDocument --> Success[Document Confirmed]  
SetRejected --> End  
Success --> End
```

Key Steps:

Teacher marks attendance with proof document

System uploads and validates file

System saves file to uploads/attendance/ directory

System creates attendance record with proof_path (proof stored in attendance record)

Admin/PIC/IB reviews proof document via confirm document endpoint

Admin/PIC/IB confirms or rejects document

If confirmed: System sets document_confirmed=true and records confirmation details

If rejected: System can update status or leave as is

API Endpoints:

POST /attendance/bulk-with-proof - Mark attendance with proof

PUT /attendance/:id/confirm-document - Confirm document (Admin)

Database Tables:

attendance - Attendance records with proof_path

admin_action_snapshots - Undo tracking

-

flowchart TD

```
Start([Generate Monthly Fees]) --> CheckScheduler{Scheduler Triggered<br/>or Manual?}
CheckScheduler -->|Scheduler| GetCurrentMonth[Get Current Month<br/>and Year]
CheckScheduler -->|Manual| AdminSelects[Admin Selects<br/>Month/Year]
GetCurrentMonth --> GetActiveStudents[Get All Active Students<br/>with Classes]
AdminSelects --> GetActiveStudents
GetActiveStudents --> ForEachStudent[For Each Student]
ForEachStudent --> CheckExistingFee{Fee Already<br/>Exists for Month?}
CheckExistingFee -->|Yes| SkipStudent[Skip Student<br/>Fee Already Generated]
CheckExistingFee -->|No| GetClassYuran[Get Class Yuran<br/>from classes table]
GetClassYuran --> CreateFee[Create Fee]
Record<br/>jumlah=class.yuran<br/>status='tunggak'<br/>bulan=current
month<br/>tahun=current year]
CreateFee --> NextStudent[Next Student]
SkipStudent --> NextStudent
NextStudent --> MoreStudents{More Students?}
MoreStudents -->|Yes| ForEachStudent
MoreStudents -->|No| SyncFees[Sync Unpaid Fees<br/>with Current Class Yuran]
SyncFees --> Success[Monthly Fees Generated]
Success --> End([End])
```

Key Steps:

System triggers monthly fee generation (scheduler or manual)

System gets all active students with assigned classes

For each student:

System checks if fee already exists for the month

If not exists: System gets class yuran (fee amount)

System creates fee record with status='tunggak' (unpaid)

System syncs unpaid fees with current class yuran (in case class fee changed)

Success response returned

API Endpoint: POST /fees/generate-monthly (Admin only)

Scheduler: Runs automatically on 1st of each month

Database Tables:

fees - Fee records

classes - Class yuran (fee amount)

students - Student enrollment

-

flowchart TD

```
Start([Process Fee Payment]) --> CheckPermission{Has Permission?<br/>Admin/Student}
CheckPermission -->|No| ShowError[Show Error]
CheckPermission -->|Yes| ValidateFee{Fee Record<br/>Exists?}
ValidateFee -->|No| ShowFeeError[Show Error:<br/>Fee not found]
ValidateFee -->|Yes| CheckOwnership{Student accessing<br/>own fee?}
CheckOwnership -->|No| CheckAdmin{Is Admin?}
CheckAdmin -->|No| ShowOwnershipError[Show Error:<br/>Access denied]
CheckAdmin -->|Yes| ProcessPayment
CheckOwnership -->|Yes| ProcessPayment[Process Payment]
ProcessPayment --> UploadReceipt{Receipt<br/>Uploaded?}
UploadReceipt -->|Yes| SaveReceipt[Save Receipt Image<br/>to uploads/receipts/]
UploadReceipt -->|No| GenerateReceiptNo[Generate Receipt Number<br/>if not provided]
SaveReceipt --> UpdateFee[Update Fee]
Record:<br/>status='terbayar'<br/>tarikh_bayar=current date<br/>cara_bayar=payment
method<br/>no_resit=receipt number<br/>resit_img=receipt path]
```

```
GenerateReceiptNo --> UpdateFee
UpdateFee --> SendConfirmation[Send Payment Confirmation<br/>Email to Student]
SendConfirmation --> CreateSnapshot[Create Admin Action<br/>Snapshot]
CreateSnapshot --> Success[Payment Processed Successfully]
Success --> End([End])
```

Key Steps:

User (Admin or Student) initiates payment

System validates fee record exists

System checks ownership (student can only pay own fees, admin can pay any)

User uploads receipt (optional) or system generates receipt number

System saves receipt image to uploads/receipts/ if provided

System updates fee record:

status='terbayar' (paid)

tarikh_bayar = current date

cara_bayar = payment method

no_resit = receipt number

resit_img = receipt image path

System sends payment confirmation email to student

System creates admin action snapshot

Success response returned

API Endpoint: PUT /fees/:id (Admin/Student)

Database Tables:

fees - Fee payment records

admin_action_snapshots - Undo tracking

Payment Methods:

Tunai (Cash)

Bank Transfer

Online Payment

-

flowchart TD

```
Start([PIC Approval Request]) --> CheckPICRole{User has<br/>PIC role?}
CheckPICRole -->|No| ShowError[Show Error:<br/>PIC role required]
ShowError --> End([End])
CheckPICRole -->|Yes| ViewPendingFees[View Pending Fees<br/>Requiring Approval]
ViewPendingFees --> SelectFee[PIC Selects Fee<br/>to Approve]
SelectFee --> ReviewPayment[Review Payment Details:<br/>Amount, Receipt, Student Info]
ReviewPayment --> PICDecision{PIC Decision}
PICDecision -->|Confirm| ConfirmDocument[Confirm
Document<br/>document_confirmed=true<br/>confirmed_by=PIC IC<br/>confirmed_at=current
timestamp]
PICDecision -->|Reject| SetRejected[Set document_confirmed=false<br/>or Leave as is]
ConfirmDocument --> UpdateFeeStatus[Update Fee Status if needed<br/>status='terbayar']
UpdateFeeStatus --> CreateSnapshot[Create Admin Action<br/>Snapshot]
SetRejected --> CreateSnapshot
CreateSnapshot --> Success[Approval Processed]
Success --> End
```

Key Steps:

PIC/IB user logs in and views fees with receipts

PIC/IB reviews payment details (amount, receipt, student info)

PIC/IB confirms document via confirm-document endpoint

If confirmed:

System sets document_confirmed=true

System records confirmation (confirmed_by, confirmed_at)

Fee status may be updated to 'terbayar' if not already

System creates admin action snapshot

Success response returned

API Endpoint: POST /fees/:id/confirm-document (PIC/IB/Admin role)

Note: PIC approval is handled through document confirmation endpoint. PIC users can confirm fee documents which effectively approves the payment.

Database Tables:

fees - Fee approval tracking (document_confirmed, confirmed_by, confirmed_at fields)

user_roles - PIC role validation

admin_action_snapshots - Undo tracking

-

flowchart TD

```
Start([Confirm Fee Document]) --> CheckPermission{Has Permission?<br/>Admin}
CheckPermission -->|No| ShowError[Show Error]
CheckPermission -->|Yes| GetFee[Get Fee Record<br/>with Receipt]
GetFee --> ReviewReceipt[Review Receipt Image<br/>and Details]
ReviewReceipt --> AdminDecision{Admin Decision}
AdminDecision -->|Confirm| UpdateConfirmation[Update Fee
Record:<br/>document_confirmed=true<br/>confirmed_by=admin IC<br/>confirmed_at=current
timestamp]
AdminDecision -->|Reject| SetRejected[Set document_confirmed=false<br/>Add rejection notes]
UpdateConfirmation --> CreateSnapshot[Create Admin Action<br/>Snapshot]
SetRejected --> CreateSnapshot
CreateSnapshot --> Success[Document Confirmation Updated]
Success --> End([End])
```

Key Steps:

Admin views fee with receipt document

Admin reviews receipt image and payment details

Admin confirms or rejects document

If confirmed:

System sets document_confirmed=true

System records admin confirmation (confirmed_by, confirmed_at)

If rejected:

System sets document_confirmed=false

System adds rejection notes

System creates admin action snapshot

Success response returned

API Endpoint: PUT /fees/:id/confirm-document (Admin only)

Database Tables:

fees - Document confirmation tracking

admin_action_snapshots - Undo tracking

-

flowchart TD

```
Start([Admin Creates Exam]) --> ValidateInput[Validate Input:<br/>Class ID, Subject, Exam Date]
ValidateInput -->|Invalid| ShowError[Show Validation Error]
ShowError --> Start
ValidateInput -->|Valid| ValidateClass{Class<br/>Exists?}
ValidateClass -->|No| ShowClassError[Show Error:<br/>Class not found]
ShowClassError --> Start
ValidateClass -->|Yes| CreateExam[Create Exam Record<br/>in exams table]
CreateExam --> CreateSnapshot[Create Admin Action<br/>Snapshot]
CreateSnapshot --> Success[Exam Created Successfully]
Success --> End([End])
```

Key Steps:

Admin enters exam details (class ID, subject, exam date)

System validates all input fields

System validates class exists

System creates exam record

System creates admin action snapshot

Success response returned

API Endpoint: POST /exams (Admin only)

Database Tables:

exams - Exam records

classes - Class validation

admin_action_snapshots - Undo tracking

-

flowchart TD

```
Start([Enter Exam Result]) --> CheckPermission{Has Permission?<br/>Admin/Teacher}
CheckPermission -->|No| ShowError[Show Error]
CheckPermission -->|Yes| ValidateInput[Validate Input:<br/>Student IC, Exam ID, Marks]
ValidateInput -->|Invalid| ShowValidationError[Show Validation Error]
```

```

ShowValidationError --> Start
ValidateInput -->|Valid| ValidateStudent{Student<br/>Exists?}
ValidateStudent -->|No| ShowStudentError[Show Error]
ValidateStudent -->|Yes| ValidateExam{Exam<br/>Exists?}
ValidateExam -->|No| ShowExamError[Show Error]
ValidateExam -->|Yes| CheckDuplicate{Result Already<br/>Exists?}
CheckDuplicate -->|Yes| ShowDuplicateError[Show Error:<br/>Result already exists]
CheckDuplicate -->|No| ValidateMarks{Validate Marks<br/>0-100}
ValidateMarks -->|Invalid| ShowMarksError[Show Error]
ValidateMarks -->|Valid| CalculateGrade[Calculate Grade<br/>Based on Settings]
CalculateGrade --> UploadSlip{Result Slip<br/>Uploaded?}
UploadSlip -->|Yes| SaveSlip[Save Slip Image<br/>to uploads/results/]
UploadSlip -->|No| CreateResult[Create Result Record:<br/>student_ic, exam_id, markah, gred,
slip_img, catatan]
SaveSlip --> CreateResult
CreateResult --> CreateSnapshot[Create Admin Action<br/>Snapshot]
CreateSnapshot --> Success[Result Entered Successfully]
Success --> End([End])

```

Key Steps:

Admin/Staff enters exam result

System validates input (student IC, exam ID, marks)

System validates student exists

System validates exam exists

System checks if result already exists for student+exam

System validates marks (0-100)

System calculates grade based on grade ranges from settings

User optionally uploads result slip image

System saves slip image to uploads/results/ if provided

System creates result record

System creates admin action snapshot

Success response returned

API Endpoint: POST /results (Admin/Staff)

Note: Teacher role is not explicitly listed in routes, but staff role includes teachers.

Database Tables:

results - Result records

exams - Exam validation

students - Student validation

settings - Grade range configuration

admin_action_snapshots - Undo tracking

-

flowchart TD

```
Start([Calculate Grade]) --> GetGradeRanges[Get Grade Ranges<br/>from Settings]
GetGradeRanges --> GetMarks[Get Student Marks<br/>0-100]
GetMarks --> CheckRanges{Check Marks<br/>Against Ranges}
CheckRanges -->|90-100| GradeA[Grade = 'A']
CheckRanges -->|80-89| GradeB[Grade = 'B']
CheckRanges -->|70-79| GradeC[Grade = 'C']
CheckRanges -->|60-69| GradeD[Grade = 'D']
CheckRanges -->|50-59| GradeE[Grade = 'E']
CheckRanges -->|0-49| GradeF[Grade = 'F']
GradeA --> ReturnGrade[Return Calculated Grade]
GradeB --> ReturnGrade
GradeC --> ReturnGrade
GradeD --> ReturnGrade
GradeE --> ReturnGrade
GradeF --> ReturnGrade
ReturnGrade --> End([End])
```

Key Steps:

System retrieves grade ranges from settings table

System gets student marks (0-100)

System compares marks against grade ranges:

A: 90-100

B: 80-89

C: 70-79

D: 60-69

E: 50-59

F: 0-49

System assigns grade based on marks

System stores grade in result record

Grade Ranges: Configurable in settings table

Database Tables:

settings - Grade range configuration

results - Grade storage

-

flowchart TD

Start([Admin Dashboard]) --> ViewOptions[Admin Can Access:]

ViewOptions --> ManageUsers[Manage Users:
Create, Update, Delete
Students, Teachers, PIC]

ViewOptions --> ManageClasses[Manage Classes:
Create, Update, Delete
Assign Teachers]

ViewOptions --> ManageAttendance[Manage Attendance:
View All, Mark, Approve
Confirm Documents]

ViewOptions --> ManageFees[Manage Fees:
Generate, View All, Update
Confirm Documents]

ViewOptions --> ManageExams[Manage Exams:
Create, Update, Delete]

ViewOptions --> ManageResults[Manage Results:
Enter, Update, Delete]

ViewOptions --> SystemSettings[System Settings:
Configure Grade Ranges
Payment Gateway
Email/SMS Settings]

ViewOptions --> ViewReports[View Reports:
Attendance Reports
Fee Reports
Result Reports]

ViewOptions --> UndoActions[Undo Actions:
View Recent Actions
Undo if Needed]

ManageUsers --> End([End])

ManageClasses --> End

ManageAttendance --> End

ManageFees --> End

ManageExams --> End

ManageResults --> End

SystemSettings --> End

ViewReports --> End

UndoActions --> End

Admin Capabilities:

Full CRUD operations on all entities

System configuration and settings

View all data (no restrictions)

Generate reports

Undo recent actions (within TTL window)

Approve/reject documents

Manage user roles and permissions

API Endpoints: All endpoints with requireRole(['admin']) middleware

-

flowchart TD

Start([Teacher Dashboard]) --> ViewOptions[Teacher Can Access:]

ViewOptions --> ViewClasses[View Assigned Classes:
List of Classes
Class Details]

ViewOptions --> ManageAttendance[Manage Attendance:
Mark Attendance
View Class Attendance
Bulk Mark with Proof]

ViewOptions --> ViewStudents[View Students:
Students in Assigned Classes
Student Profiles]

ViewOptions --> EnterResults[Enter Exam Results:
For Students in Classes]

ViewOptions --> ViewResults[View Results:
Results for Class Students]

ViewOptions --> ViewFees[View Fees:
Fees for Class Students
Read-only]

ViewClasses --> End([End])

ManageAttendance --> End

ViewStudents --> End

EnterResults --> End

ViewResults --> End

ViewFees --> End

Teacher Capabilities:

View and manage assigned classes only

Mark attendance for their classes

Enter exam results for their students (if staff role includes teachers)

View student profiles (in their classes)

View fees (read-only, for their students)

Cannot create/delete classes

Cannot manage users

Cannot access system settings

API Endpoints: Endpoints with role-based filtering (teacher can only access their classes)

-

flowchart TD

Start([Student Dashboard]) --> ViewOptions[Student Can Access:]

ViewOptions --> ViewProfile[View Own Profile:
Personal Information
Class

Assignment]

ViewOptions --> UpdateProfile[Update Own Profile:
Email, Phone, Address
Cannot change IC/Name]

ViewOptions --> ViewAttendance[View Own Attendance:
Attendance

History
Attendance Statistics]

ViewOptions --> ViewFees[View Own Fees:
Fee History
Payment Status
Pay Fees]

ViewOptions --> ViewResults[View Own Results:
Exam Results
Grades]

ViewOptions --> ViewAnnouncements[View Announcements:
Targeted to Students]

ViewProfile --> End([End])

UpdateProfile --> End

ViewAttendance --> End

ViewFees --> End

ViewResults --> End

ViewAnnouncements --> End

Student Capabilities:

View own profile and information

Update own profile (limited fields)

View own attendance history

View own fees and make payments

View own exam results

View announcements

Cannot access other students' data

Cannot mark attendance

Cannot enter results

Cannot manage classes

API Endpoints: Endpoints with ownership filtering (student can only access own data)

-

flowchart TD

```
Start([IB/PIC Dashboard]) --> ViewOptions[IB Can Access:]
ViewOptions --> ViewPendingFees[View Pending Fees:<br/>Fees Requiring Approval<br/>With Receipt Documents]
ViewOptions --> ApproveFees[Approve Fees:<br/>Review Receipt<br/>Approve Payment]
ViewOptions --> RejectFees[Reject Fees:<br/>If Receipt Invalid<br/>Add Rejection Notes]
ViewOptions --> ViewFeeHistory[View Fee History:<br/>All Approved/Rejected Fees]
ViewPendingFees --> End([End])
ApproveFees --> End
RejectFees --> End
ViewFeeHistory --> End
```

IB/PIC Capabilities:

View pending fees requiring approval

Review payment receipts

Approve or reject fee payments

View fee approval history

Cannot create/update fees

Cannot manage students/classes

Cannot access attendance/results

API Endpoints:

GET /fees - View fees (can filter by status)

POST /fees/:id/confirm-document - Confirm fee document (approves payment)

GET /fees/:id - View specific fee details

Database Tables:

fees - Fee approval tracking

user_roles - PIC role validation

-

flowchart TD

```
Start([Yearly Migration Process]) --> AdminInitiates[Admin Initiates<br/>New Year Migration]
AdminInitiates --> BackupCurrentDB[Backup Current Year<br/>Database]
BackupCurrentDB --> CreateNewDB[Create New Year Database<br/>masjid_app_YYYY]
CreateNewDB --> RunSchema[Run Database Schema<br/>Create All Tables]
RunSchema --> TransferData[Transfer Data from<br/>Old Year to New Year]
TransferData --> CopyStudents[Copy Students Table<br/>All Student Records]
TransferData --> CopyTeachers[Copy Teachers Table<br/>All Teacher Records]
TransferData --> CopyClasses[Copy Classes Table<br/>All Class Records]
CopyStudents --> ResetTransactional[Reset Transactional Data:<br/>Clear Fees, Attendance,
Results]
CopyTeachers --> ResetTransactional
CopyClasses --> ResetTransactional
ResetTransactional --> UpdateMasterDB[Update Master Database<br/>Set New Year as Active]
UpdateMasterDB --> VerifyMigration[Verify Migration<br/>Check Data Integrity]
VerifyMigration --> Success{Migration<br/>Successful?}
Success -->|Yes| ActivateNewYear[Activate New Year<br/>Update Application Config]
Success -->|No| Rollback[Rollback Migration<br/>Restore from Backup]
ActivateNewYear --> End([End])
Rollback --> End
```

Key Steps:

Admin initiates yearly migration process

System backs up current year database

System creates new year database: masjid_app_YYYY

System runs database schema to create all tables

System transfers persistent data:

Students table (all records)

Teachers table (all records)

Classes table (all records)

System resets transactional data:

Fees table (cleared)

Attendance table (cleared)

Results table (cleared)

Exams table (cleared)

System updates master database to set new year as active

System verifies migration (data integrity checks)

If successful: System activates new year

If failed: System rolls back from backup

API Endpoint: POST /migration/migrate-year (Admin only)

Database Structure:

Master DB: masjid_master - Tracks active years

Year DBs: masjid_app_2024, masjid_app_2025, etc.

Benefits:

Clean yearly separation

Easy backup and restore

Historical data preservation

Simplified queries (no year filtering needed)

-

flowchart TD

Start([Admin Performs Action]) --> ActionType{Action Type}

ActionType -->|Create| CreateEntity[Create Entity
Student/Class/Teacher/etc]

ActionType -->|Update| UpdateEntity[Update Entity]

ActionType -->|Delete| DeleteEntity[Delete Entity]

CreateEntity --> CaptureSnapshot[Capture Snapshot:
Operation, Entity Type,
Entity ID, Data, Actor IC]

UpdateEntity --> CaptureSnapshot

DeleteEntity --> CaptureSnapshot

CaptureSnapshot --> StoreSnapshot[Store Snapshot in
admin_action_snapshots table]

StoreSnapshot --> SetExpiry[Set Expiry Time
TTL: 24 hours]

SetExpiry --> ContinueAction[Continue with Action]

ContinueAction --> Success[Action Completed]

Success --> AdminViewsUndo[Admin Views Undoable Actions
Within TTL Window]

AdminViewsUndo --> SelectUndo{Select Action
to Undo?}

SelectUndo -->|Yes| ValidateUndo{Validate Undo:
Not Already Undone
Not Expired}

ValidateUndo -->|Invalid| ShowUndoError[Show Error:
Cannot Undo]

ValidateUndo -->|Valid| ExecuteUndo[Execute Undo Handler
Based on Entity Type]

ExecuteUndo --> RestoreData[Restore/Reverse Data
Based on Operation]

RestoreData --> MarkUndone[Mark Snapshot
was_undone=true]

```
MarkUndone --> UndoSuccess[Undo Successful]
SelectUndo -->|No| End([End])
ShowUndoError --> End
UndoSuccess --> End
```

Key Steps:

Admin performs action (create/update/delete) on entity

System captures snapshot before action:

Operation type (create/update/delete)

Entity type (student/class/teacher/etc)

Entity ID/identifier

Full data snapshot (before state for update/delete, after state for create)

Actor IC (admin who performed action)

System stores snapshot in admin_action_snapshots table

System sets expiry time (TTL: 24 hours default)

System continues with original action

Admin can view undoable actions (within TTL window)

Admin selects action to undo

System validates undo (not already undone, not expired)

System executes entity-specific undo handler

System restores/reverses data based on operation:

Create → Delete

Update → Restore previous state

Delete → Restore entity

System marks snapshot as undone

Undo successful

Supported Entity Types:

Student

Teacher

Class

Attendance

Fee

Exam

Result

Announcement

PIC User

API Endpoints:

GET /admin-actions - List undoable actions

POST /admin-actions/:id/undo - Undo action

Database Tables:

admin_action_snapshots - Snapshot storage

All entity tables (for undo operations)

Snapshot TTL: 24 hours (configurable via SNAPSHOT_TTL_HOURS)

-

POST /auth/register - Student self-registration

POST /auth/login - User login

POST /auth/check-reset-options - Check password reset options

POST /auth/request-reset-email - Request email password reset

POST /auth/request-reset-phone - Request phone password reset

POST /auth/reset-password - Reset password with token/code

GET /students - List students

GET /students/:ic - Get student by IC

POST /students - Create student (Admin)

PUT /students/:ic - Update student

DELETE /students/:id - Delete student (Admin)

GET /classes - List classes

GET /classes/:id - Get class by ID

POST /classes - Create class (Admin)

PUT /classes/:id - Update class (Admin)

DELETE /classes/:id - Delete class (Admin)

GET /attendance - List attendance records

POST /attendance - Mark individual attendance

POST /attendance/bulk - Mark bulk attendance

POST /attendance/bulk-with-proof - Mark attendance with proof

PUT /attendance/:id/confirm-document - Confirm document (Admin)

DELETE /attendance/:id - Delete attendance (Admin)

GET /fees - List fees

GET /fees/:id - Get fee by ID

POST /fees - Create fee (Admin)

POST /fees/generate-monthly - Generate monthly fees (Admin)

PUT /fees/:id - Update fee (payment)

POST /fees/:id/confirm-document - Confirm fee document (PIC/IB/Admin)

PUT /fees/:id/confirm-document - Confirm document (Admin)

DELETE /fees/:id - Delete fee (Admin)

GET /exams - List exams

GET /exams/:id - Get exam by ID

POST /exams - Create exam (Admin)

PUT /exams/:id - Update exam (Admin)

DELETE /exams/:id - Delete exam (Admin)

GET /results - List results

GET /results/:id - Get result by ID

POST /results - Create result (Admin/Staff)

PUT /results/:id - Update result (Admin/Staff)

DELETE /results/:id - Delete result (Admin)

GET /admin-actions - List undoable actions

POST /admin-actions/:id/undo - Undo action

POST /migration/migrate-year - Migrate to new year (Admin)

-

users - All users (students, teachers, admins, PIC)

students - Student-specific data (class assignment)

teachers - Teacher-specific data (expertise)

classes - Class information (schedule, fee, teacher)

user_roles - User role assignments (RBAC)

attendance - Attendance records

fees - Fee records and payments

exams - Exam records

results - Exam result records

admin_action_snapshots - Undo tracking

password_reset_tokens - Password reset tokens/codes

login_attempts - Failed login tracking

settings - System configuration

-

Authentication & Authorization

JWT-based authentication

Role-based access control (RBAC)

Account lockout after failed attempts

Password hashing (bcrypt, 12 rounds)

Password Recovery

Two methods: Email (24h) and SMS (10min)

One-time use tokens/codes

User enumeration prevention

Data Protection

Input validation and sanitization

SQL injection prevention (parameterized queries)

File upload validation

Role-based data filtering

Audit Trail

Admin action snapshots

Undo functionality (24h TTL)

Login attempt tracking

-

All workflows support error handling and validation

Admin actions can be undone within 24 hours

Role-based access control enforced at API level

Yearly database migration preserves historical data

File uploads stored in uploads/ directory with subdirectories

Email/SMS services optional (graceful degradation in development)

-

Last Updated: 2025

Deployment Guide

✅ Data centers in Malaysia: Cyberjaya & Penang (lowest latency!)

✅ 24/7 Support: Bahasa Melayu & English

- ✓ Docker support: Full Docker Compose support
- ✓ Local payment: Bank transfer, credit card (MYR)
- ✓ Pricing: RM 30-80/month (~\$7-18 USD)
- 🌐 Website: [exabytes.com](https://www.exabytes.com)
- 💡 Best for: Local support, low latency, Malaysian payment methods
- ✓ Data center: Singapore (very low latency to Malaysia ~5-10ms)
- ✓ Full Docker Compose support
- ✓ Easy setup: Excellent documentation
- ✓ Pricing: \$6-12/month (~RM 28-56)
- 🌐 Website: [digitalocean.com](https://www.digitalocean.com)
- 💡 Best for: Reliability, global scale, easy management
- 📌 Important: Select Singapore region when creating droplet!
- ✓ VPS with Docker pre-installed
- ✓ Asian data centers (good latency)
- ✓ Pricing: RM 25-60/month
- 🌐 Website: [hostinger.com/my](https://www.hostinger.com/my)
- 💡 Best for: Easy setup, pre-configured Docker
- ✓ Local Malaysian provider
- ✓ VPS & Dedicated servers
- ✓ Responsive local support
- ✓ Pricing: RM 40-100/month
- 🌐 Website: [serverfreak.com](https://www.serverfreak.com)
- 💡 Best for: Local support, customizable plans
- ✓ Data center: Singapore

✓ Managed by AWS

✓ Pricing: \$3.50-10/month (~RM 16-46)

🌐 Website: aws.amazon.com/lightsail

💡 Best for: AWS ecosystem, scalable

✓ Data center: Singapore

✓ Pay-as-you-go

✓ Pricing: \$6-12/month

🌐 Website: [vultr.com](https://www.vultr.com)

💡 Best for: Flexible pricing, good performance

•

For Best Local Support & Lowest Latency: → Exabytes Malaysia (Cyberjaya/Penang data centers)

For Best Global Service & Reliability: → DigitalOcean Singapore (select Singapore region!)

For Easiest Setup: → Hostinger Malaysia (Docker pre-installed)

•

Why DigitalOcean Droplet (Singapore)?

✓ Full Docker Compose support (runs exactly as local)

✓ Complete control over your environment

✓ Persistent storage for uploads and database

✓ Easy SSL setup with Let's Encrypt

✓ Cost-effective (\$6-12/month for small apps)

✓ Singapore region = Very low latency to Malaysia (~5-10ms)

✓ Simple to scale up when needed

✓ No vendor lock-in

✓ Excellent documentation & community

Alternative Options:

Exabytes Malaysia - Local provider, best latency, Malay support (RM 30-80/month)

Hostinger Malaysia - Docker pre-installed, easy setup (RM 25-60/month)

Railway - Easiest Docker deployment, auto-deploy from Git (\$5-20/month)

Render - Managed services, good for beginners (\$7-25/month)

AWS Lightsail Singapore - Similar to DigitalOcean (\$3.50-10/month)

-
- ☐ Domain name purchased and DNS access
- ☐ Email account with app password (Gmail/Outlook)
- ☐ Twilio account (for SMS) - Optional
- ☐ Strong passwords generated for database and JWT
- ☐ Backup strategy planned
-

Sign up at [digitalocean.com](https://www.digitalocean.com)

Create a Droplet:

Image: Ubuntu 22.04 LTS

Plan: Basic (\$6/month - 1GB RAM) or Regular (\$12/month - 2GB RAM)

Region: ⚠️ IMPORTANT - Select SINGAPORE (lowest latency to Malaysia!)

Authentication: SSH keys (recommended) or password

Hostname: mymasjidapp or your domain

SSH into your server

```
ssh root@your_server_ip
```

Update system

```
apt update && apt upgrade -y
```

Install Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sh get-docker.sh
```

Install Docker Compose

```
apt install docker-compose-plugin -y
```

Verify installation

```
docker --version
```

docker compose version

Create app directory

mkdir -p /opt/mymasjidapp

cd /opt/mymasjidapp

Install Git

apt install git -y

Clone your repository

git clone https://github.com/yourusername/MyMasjidApp.git .

Option A: Using Git (Recommended)

From your local machine

scp -r . root@your_server_ip:/opt/mymasjidapp/

Option B: Using SCP (from your local machine)

Create production .env files

cd /opt/mymasjidapp

Copy and edit backend .env

cp backend/env.production backend/.env

nano backend/.env

Copy and edit root .env

cp env.production .env

nano .env

Required Environment Variables:

DB_PASSWORD - Strong database password

JWT_SECRET - Random 32+ character string

FRONTEND_URL - Your domain (https://yourdomain.com)

EMAIL_USER - Your email

EMAIL_PASSWORD - App password

TWILIO_ - If using SMS

Install Certbot

apt install certbot -y

Stop nginx temporarily

docker compose down

Get SSL certificate

certbot certonly --standalone -d yourdomain.com -d www.yourdomain.com

Certificates will be in:

/etc/letsencrypt/live/yourdomain.com/fullchain.pem

/etc/letsencrypt/live/yourdomain.com/privkey.pem

Copy SSL certificates to project

mkdir -p nginx/ssl

cp /etc/letsencrypt/live/yourdomain.com/fullchain.pem nginx/ssl/cert.pem

cp /etc/letsencrypt/live/yourdomain.com/privkey.pem nginx/ssl/key.pem

Update nginx.conf with your domain

nano nginx/nginx.conf

Change "yourdomain.com" to your actual domain

Use the production docker-compose file

cp docker-compose.prod.yml docker-compose.yml

Or edit existing docker-compose.yml

Build and start services

docker compose build --no-cache

docker compose up -d

Check status

docker compose ps

View logs

docker compose logs -f

Wait for MySQL to be ready (30 seconds)

sleep 30

Run migrations

docker compose exec backend npm run migrate

Test renewal

certbot renew --dry-run

Add to crontab (auto-renew every 12 hours)

crontab -e

Add this line:

0 /12 certbot renew --quiet && docker compose restart nginx

```
# Install UFW
apt install ufw -y
```

```
# Allow SSH, HTTP, HTTPS
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
```

```
# Enable firewall
ufw enable
```

```
# Create backup script
nano /opt/mymasjidapp/backup.sh
```

Add backup script content (see backup section below).

-

Sign up at [exabytes.com](https://www.exabytes.com)

Choose VPS Plan:

Select VPS with Ubuntu 22.04 LTS

Choose Cyberjaya or Penang data center (lowest latency!)

Minimum: 1GB RAM, 1 CPU core

Pricing: RM 30-80/month

You'll receive server IP and root password via email

SSH into server:

```
ssh root@your_server_ip
# Enter password when prompted
```

Continue with Step 2 onwards from the DigitalOcean section above. The setup process is identical!

Advantages of Exabytes:

- ✅ Support in Bahasa Melayu
- ✅ Local payment methods (Bank transfer, FPX)
- ✅ Data centers in Malaysia (lowest latency)
- ✅ Local business hours support

-

Sign up at railway.app

Install Railway CLI:

```
npm i -g @railway/cli  
railway login
```

In your project directory

```
railway init  
railway link
```

Add MySQL Database:

In Railway dashboard: New → Database → MySQL

Note the connection string

Add Backend Service:

New → GitHub Repo → Select your repo

Root Directory: /backend

Build Command: npm ci

Start Command: npm start

Add environment variables from backend/env.production

Add Frontend Service:

New → GitHub Repo → Select your repo

Root Directory: /

Build Command: npm ci && npm run build

Start Command: npm run preview

Add environment variables from env.production

In Railway dashboard, add all variables from:

backend/env.production

env.production

Update DB_HOST to Railway's MySQL host

Update FRONTEND_URL to Railway's frontend URL

railway up

Railway will automatically deploy on every git push!

-

Sign up at render.com

New → PostgreSQL/MySQL → MySQL

Note connection details

New → Web Service

Connect GitHub repo

Settings:

Name: mymasjidapp-backend

Root Directory: backend

Build Command: npm ci

Start Command: npm start

Environment: Node

Plan: Free or Starter (\$7/month)

New → Static Site

Connect GitHub repo

Settings:

Build Command: npm ci && npm run build

Publish Directory: dist

Add all variables in Render dashboard for each service.

-

Generate strong passwords

openssl rand -base64 32

Only expose ports 80, 443, and 22 (SSH)

Use fail2ban for SSH protection

Set up automatic security updates

```
apt install unattended-upgrades -y  
dpkg-reconfigure -plow unattended-upgrades
```

Use strong database passwords

Restrict database access to localhost only

Regular backups

-

Create /opt/mymasjidapp/backup.sh:

```
#!/bin/bash  
BACKUP_DIR="/opt/backups/mymasjidapp"  
DATE=$(date +%Y%m%d_%H%M%S)  
mkdir -p $BACKUP_DIR  
  
# Backup database  
docker compose exec -T mysql mysqldump -u root -p$DB_PASSWORD masjid_app >  
$BACKUP_DIR/db_$DATE.sql  
  
# Backup uploads  
tar -czf $BACKUP_DIR/uploads_$DATE.tar.gz uploads/  
  
# Keep only last 7 days  
find $BACKUP_DIR -type f -mtime +7 -delete  
  
# Optional: Upload to cloud storage (S3, etc.)  
  
chmod +x /opt/mymasjidapp/backup.sh
```

Make it executable:

```
crontab -e  
# Add:  
0 2 /opt/mymasjidapp/backup.sh
```

Add to crontab (daily at 2 AM):

-

SSH into server

```
ssh root@your_server_ip
```

```
cd /opt/mymasjidapp
```

```
# Pull latest changes
```

```
git pull
```

```
# Rebuild and restart
```

```
docker compose build --no-cache
```

```
docker compose down
```

```
docker compose up -d
```

```
# Run migrations if needed
```

```
docker compose exec backend npm run migrate
```

Just push to GitHub - Railway auto-deploys!

```
git add .
```

```
git commit -m "Update application"
```

```
git push
```

-

```
# View running containers
```

```
docker compose ps
```

```
# View logs
```

```
docker compose logs -f
```

```
# Check specific service
```

```
docker compose logs backend -f
```

Frontend: <https://yourdomain.com/health>

Backend: <https://yourdomain.com/api/health>

Consider adding:

Uptime monitoring: UptimeRobot (free)

Error tracking: Sentry (free tier)

Analytics: Google Analytics

-

```
# Check logs
```

```
docker compose logs
```

Check if ports are in use

netstat -tulpn | grep :80

netstat -tulpn | grep :443

Restart services

docker compose restart

Check MySQL is running

docker compose ps mysql

Check MySQL logs

docker compose logs mysql

Test connection

docker compose exec mysql mysql -u root -p

Renew certificate manually

certbot renew

Check certificate expiry

certbot certificates

Restart nginx

docker compose restart nginx

-

DigitalOcean Docs: <https://docs.digitalocean.com>

Docker Docs: <https://docs.docker.com>

Let's Encrypt: <https://letsencrypt.org/docs/>

-

- ☐ Application accessible via domain

- ☐ SSL certificate working (HTTPS)

- ☐ Database migrations completed

- ☐ Admin account created

- ☐ Email sending working

- ☐ File uploads working

- ☐ Backups configured

- ☐ Monitoring set up

- ☐ Firewall configured

- ☐ Documentation updated

-

Need Help? Check the troubleshooting section or review the logs for specific error messages.

Development Standards

Version 1.0 Date: January 2025 ---

This development standards manual establishes the technical guidelines, processes, and best practices for developing and maintaining the MyMasjidApp system. It ensures consistency, quality, and maintainability across the development lifecycle.

This manual applies to:

All software development activities for MyMasjidApp

Frontend development (React)

Backend development (Node.js/Express)

Database design and management (MySQL)

DevOps and deployment processes

Quality assurance and testing

Documentation requirements

This manual is intended for:

Software developers working on the MyMasjidApp codebase

Quality assurance engineers and testers

DevOps engineers and system administrators

Project managers and technical leads

New team members joining the project

API: Application Programming Interface - the interface between frontend and backend

CI/CD: Continuous Integration / Continuous Deployment - automated build and deployment processes

Docker: Containerization platform used for deployment

JWT: JSON Web Token - authentication mechanism

MVC: Model-View-Controller - architectural pattern

SDLC: Software Development Lifecycle

VPS: Virtual Private Server - hosting environment

ORM: Object-Relational Mapping - database abstraction layer

-

MyMasjidApp follows a Hybrid Agile/DevOps approach combining:

Agile principles: Iterative development, user feedback, flexible planning

DevOps practices: Continuous integration, automated deployment, infrastructure as code

Incremental delivery: Regular releases with new features and improvements

Activities:

Collect user requirements through stakeholder meetings

Document functional and non-functional requirements

Define acceptance criteria for each feature

Prioritize features using backlog management

Update requirements based on feedback

Deliverables:

Feature specification documents

User stories with acceptance criteria

Requirements traceability matrix

Activities:

Create system architecture diagrams

Design database schema and relationships

Plan API endpoints and data models

Design user interface mockups (if applicable)

Define security and authentication flows

Document integration points

Deliverables:

Architecture documentation

Database schema design

API specification

UI/UX designs (if applicable)

Activities:

Write code following coding standards (see Section 3)

Implement features according to specifications

Write inline documentation and comments

Follow version control guidelines (see Section 4)

Conduct peer code reviews

Update documentation as code evolves

Deliverables:

Source code

Code documentation

Unit tests

Updated API documentation

Activities:

Write and execute unit tests

Perform integration testing

Conduct system testing

Execute user acceptance testing

Perform security testing

Report and track bugs

Deliverables:

Test cases and test results

Bug reports

Test coverage reports

Quality metrics

Activities:

Build application for production

Run deployment scripts

Verify deployment success

Monitor application health

Perform smoke tests in production

Document deployment process

Deliverables:

Deployed application

Deployment logs

Deployment documentation

Monitoring setup

Activities:

Monitor application performance

Fix bugs and issues

Apply security patches

Update dependencies

Add new features

Optimize performance

Maintain documentation

Deliverables:

Bug fixes

Performance improvements

Updated documentation

Release notes

-

Frontend:

JavaScript (ES6+)

React 19.1.1

JSX syntax

Backend:

JavaScript (ES6+)

Node.js (LTS version)

Express.js

Database:

MySQL 8.0

SQL syntax

Naming conventions ensure code readability and consistency across the project. Well-named variables, functions, and files make code self-documenting and easier to maintain.

Use camelCase for variables - This convention is standard in JavaScript and makes variables easy to read

Use descriptive names that indicate purpose - Clear names reduce the need for comments and make code self-explanatory

Boolean variables should start with is, has, should, or can - This makes boolean checks read like natural language (e.g., if (isActive) reads better than if (active))

Constants should use UPPER_SNAKE_CASE - Uppercase letters make constants stand out, indicating they should not be changed

```
// Good - from backend/controllers/authController.js
const SESSION_DURATION_SECONDS = 24 * 60 * 60; // 24 hours
const REFRESH_TOKEN_DURATION_SECONDS = 7 * 24 * 60 * 60; // 7 days
const normalizedIC = normalizeICForQuery(ic_number);
const hasStudentRole = existingUser.role === userRole;
const isTokenExpired = () => {
  const expiry = getStoredExpiry();
  return typeof expiry === 'number' && !Number.isNaN(expiry) && Date.now() > expiry;
};
```

```
// Bad
const u = 'John';
const active = true;
const x = 3;
```

Examples from MyMasjidApp:

Use camelCase for function names - Consistent with JavaScript conventions

Use verb-noun pattern for function names - The verb describes the action (get, create, update, delete), and the noun describes what is being acted upon (e.g., getUserById, createStudent)

Use descriptive names that indicate what the function does - Function names should clearly communicate their purpose without needing to read the implementation

Why this matters: Clear function names make code self-documenting. When a developer sees `getStudentById(id)`, they immediately understand what the function does without reading its implementation.

```
// Good - from backend/controllers/authController.js
export const register = async (req, res) => { }
export const login = async (req, res) => { }
async function attachRoleMetadata(user) { }
const isTokenExpired = () => { }
const normalizeICForQuery = (ic) => { }
const isValidICFormat = (ic) => { }
```

```
// Good - from src/services/api.js
export const authAPI = {
  login: async (data) => { },
  register: async (data) => { },
  getProfile: async () => { }
};
```

```
// Bad
function get() { }
function calc() { }
function check() { }
```

Examples from MyMasjidApp:

Use PascalCase for class names

Use camelCase for module/file names (React components use PascalCase)

Use descriptive names that indicate purpose

```
// Good - React components use PascalCase
// src/pages/Pelajar.jsx
const Pelajar = ({ user }) => { }
```

```
// src/components/auth/Login.jsx
const Login = () => { }
```

```
// Backend controllers - camelCase file names, exported functions
// backend/controllers/authController.js
export const register = async (req, res) => { }
export const login = async (req, res) => { }
```

```
// backend/controllers/studentController.js
export const getAllStudents = async (req, res) => { }
export const createStudent = async (req, res) => { }
```

```
// Bad
class service { }
class ctrl { }
```

Examples from MyMasjidApp:

Use kebab-case for file and folder names (or camelCase for JS files) - kebab-case works well across all operating systems and is URL-friendly

React component files use PascalCase - This convention matches React component naming and makes it easy to distinguish component files from utility files

Keep file names concise but descriptive - File names should be long enough to be clear but short enough to be manageable

Explanation: File naming conventions help developers quickly locate files. When you see `authController.js`, you know it's an authentication controller. When you see `Login.jsx`, you know it's a React component for login functionality.

```
// Good - Backend files (camelCase/kebab-case)
backend/routes/auth.js
backend/routes/students.js
backend/routes/teachers.js
backend/controllers/authController.js
backend/controllers/studentController.js
backend/middleware/auth.js
backend/utlis/icUtils.js
backend/config/database.js
```

```
// Good - Frontend React components (PascalCase)
src/pages/Pelajar.jsx
src/pages/Guru.jsx
src/pages/Kelas.jsx
src/components/auth/Login.jsx
src/components/auth/Register.jsx
src/components/pelajar/PelajarList.jsx
src/services/api.js
src/utls/icUtils.js
```

```
// Bad
file1.js
temp.js
user.js
```

Examples from MyMasjidApp:

Use 2 spaces for indentation (no tabs) - Two spaces provide good readability without excessive horizontal scrolling. Tabs can display differently on different systems

Configure editor to show whitespace - This helps catch inconsistent indentation and trailing spaces

Maintain consistent indentation levels - Consistent indentation makes code structure visually clear

Why this matters: Consistent indentation is crucial for readability. It helps developers understand code structure at a glance and makes debugging easier.

Maximum line length: 100 characters - This length fits comfortably on most screens and allows side-by-side code review without horizontal scrolling

Break long lines for readability - Long lines are harder to read and understand

Align continuation lines appropriately - Continuation lines should be indented to show they are part of the same statement

Explanation: Enforcing a maximum line length improves code readability, especially during code reviews or when viewing code on smaller screens.

Inline Comments:

Use comments to explain "why", not "what" - The code itself shows "what" it does. Comments should explain the reasoning, edge cases, or non-obvious behavior

Write comments in clear, concise English - Comments should be as readable as the code itself

Update comments when code changes - Outdated comments are misleading and worse than no comments

Remove commented-out code before committing - Version control (Git) keeps history, so commented code is unnecessary

Why this matters: Good comments explain the reasoning behind code decisions, making it easier for other developers (or yourself in the future) to understand why something was done a certain way.

Function Documentation:

Document all public functions and methods

Use JSDoc format for JavaScript functions

Include parameter descriptions and return types

```
// backend/controllers/authController.js
/  
Authenticates a user and returns a JWT token  
@param {Object} req - Express request object  
@param {Object} res - Express response object  
@returns {Promise<void>}  
/  
export const login = async (req, res) => {  
  try {  
    const { icNumber, password } = req.body;  
    // Implementation...  
  } catch (error) {  
    // Error handling...  
  }  
};  
  
/  
Normalizes IC number for database queries (removes hyphens and spaces)  
@param {string} ic - IC number with or without hyphens  
@returns {string} Normalized IC number (12 digits)  
/  
const normalizeICForQuery = (ic) => {  
  return typeof ic === 'string' ? ic.replace(/[-\s]/g, '') : ic;  
};
```

Examples from MyMasjidApp:

Extract common functionality into reusable functions - If the same logic appears in multiple places, extract it into a function

Create utility modules for shared code - Shared utilities (like IC normalization, phone formatting) should be in utility modules

Avoid duplicating code across files - Code duplication makes maintenance harder because bugs must be fixed in multiple places

Explanation: DRY principle reduces code duplication, making the codebase smaller, easier to maintain, and less error-prone. When logic needs to change, it only needs to be updated in one place. In MyMasjidApp, IC normalization is centralized in backend/utils/icUtils.js, so if the format changes, it only needs to be updated once.

Single Responsibility: Each function/class should have one purpose - A function that does too many things is harder to understand, test, and maintain

Open/Closed: Open for extension, closed for modification - Code should be extensible without modifying existing functionality

Liskov Substitution: Derived classes must be substitutable - Subclasses should be usable wherever their parent class is expected

Interface Segregation: Many specific interfaces are better than one general - Interfaces should be focused and not force implementations of unused methods

Dependency Inversion: Depend on abstractions, not concretions - High-level modules should not depend on low-level modules; both should depend on abstractions

Explanation: SOLID principles help create maintainable, flexible, and testable code. For example, in MyMasjidApp, authentication logic is separated from business logic (controllers depend on middleware for authentication), making it easy to change authentication methods without affecting business logic.

Always handle errors explicitly - Never ignore errors. Always handle them appropriately (log, return error response, or rethrow)

Use try-catch blocks for async operations - Async operations can fail, and try-catch blocks prevent unhandled promise rejections

Provide meaningful error messages - Error messages should help users understand what went wrong and how to fix it

Log errors appropriately - Log errors with context (user ID, request details) for debugging, but don't log sensitive information

Don't expose sensitive information in error messages - Error messages sent to users should not reveal database structure, internal paths, or other sensitive details

Why this matters: Proper error handling improves user experience and makes debugging easier. In MyMasjidApp, controllers catch errors and return user-friendly messages in Malay, while logging detailed error information for developers.

```

export const register = async (req, res) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      const firstError = errors.array()[0];
      return res.status(400).json({
        success: false,
        message: firstError.msg || 'Validation failed',
        errors: errors.array()
      });
    }

    const { nama, ic_number, email, password, telefon, umur } = req.body;

    // Validate and normalize IC number
    if (!isValidICFormat(ic_number)) {
      return res.status(400).json({
        success: false,
        message: 'Format IC tidak sah. Sila masukkan 12 digit nombor IC.'
      });
    }

    // Hash password
    const hashedPassword = await bcrypt.hash(password, 12);

    // Create user...
    return res.status(201).json({
      success: true,
      message: 'Registration successful'
    });
  } catch (error) {
    console.error('[REGISTER] Error:', error);
    return res.status(500).json({
      success: false,
      message: 'Ralat sistem. Sila cuba lagi.'
    });
  }
};

```

Example from MyMasjidApp (backend/controllers/authController.js):

Use appropriate log levels (info, warn, error, debug)

Include context in log messages

Don't log sensitive information (passwords, tokens)

Use structured logging when possible

```
// backend/middleware/securityLogger.js
import { logFailedAuthAttempt, logSuspiciousActivity } from '../middleware/securityLogger.js';
```

```
// In authController.js
console.error('[REGISTER] Password hashing error:', error);
console.warn('[REGISTER] Weak password used for IC: ${normalizedIC}`);
console.error('[REGISTER] Error checking for existing users:', error);
```

```
// Structured logging example
logger.info('User login attempt', {
  icNumber: user.ic,
  timestamp: new Date(),
  ip: req.ip
});
logger.error('Database connection failed', {
  error: error.message,
  stack: error.stack
});
```

Example from MyMasjidApp:

-

Git - Primary version control system

GitHub/GitLab/Bitbucket - Remote repository hosting (as configured)

MyMasjidApp uses a simplified Git Flow approach:

main (or master): Production-ready code

Protected branch (requires pull request)

Only stable, tested code

Each commit should be tagged with version number

develop: Integration branch for development

Main development branch

All feature branches merge here

Should always be in a deployable state

feature/[feature-name]: New features

Branch from: develop

Merge to: develop

Naming: feature/user-authentication, feature/payment-gateway

bugfix/[bug-description]: Bug fixes

Branch from: develop or main (depending on severity)

Merge to: develop or main

Naming: bugfix/login-error, bugfix/database-connection

hotfix/[issue-description]: Critical production fixes

Branch from: main

Merge to: main and develop

Naming: hotfix/security-patch, hotfix/critical-bug

Use Conventional Commits format:

<type>(<scope>): <subject>

<body>

<footer>

Types:

feat: New feature

fix: Bug fix

docs: Documentation changes

style: Code style changes (formatting, etc.)

refactor: Code refactoring

test: Test additions/changes

chore: Maintenance tasks

perf: Performance improvements

ci: CI/CD changes

feat(auth): add JWT token refresh mechanism

Implement automatic token refresh when token expires.

Token refresh happens in the background without user intervention.

Closes #123

Examples:

fix(payment): resolve fee calculation error for partial payments

Fixed incorrect fee calculation when student pays partial amount.

Updated validation logic to handle edge cases.

Fixes #456

docs(api): update API documentation for user endpoints

Added examples and error responses to user API documentation.

Branch: Create branch from appropriate base branch

Commits: Ensure commits follow commit message conventions

Tests: All tests must pass

Documentation: Update documentation if needed

Description: Provide clear PR description with:

What changes were made

Why the changes were needed

How to test the changes

Screenshots (for UI changes)

Reviewers should check:

Code follows coding standards

Logic is correct and efficient

Error handling is appropriate

Tests are adequate

Documentation is updated

No security vulnerabilities

Performance implications considered

Review Process:

Author creates pull request

At least one reviewer assigned

Reviewer reviews code and provides feedback

Author addresses feedback

Reviewer approves when satisfied

PR is merged (squash and merge preferred)

Review Checklist:

- ☐ Code follows naming conventions
- ☐ Code is properly formatted
- ☐ Functions have appropriate documentation
- ☐ Error handling is implemented
- ☐ Tests are included and passing
- ☐ No security issues
- ☐ Performance is acceptable
- ☐ Documentation is updated
-

MyMasjidApp uses a Layered Architecture with MVC pattern:

View Layer: React components (presentation)

Service Layer: API service modules (business logic)

State Management: React Context API / Hooks

Routing: React Router (navigation)

Routes Layer: API route definitions - Routes in backend/routes/ define URL patterns and map them to controllers. They also apply middleware (authentication, validation)

Controller Layer: Request handlers (business logic) - Controllers in backend/controllers/ handle HTTP requests, validate input, call services, and format responses

Service Layer: Business logic and data processing - Services in backend/services/ contain reusable business logic that can be called by multiple controllers

Model Layer: Database interactions and data models - Database queries are executed using the connection pool from backend/config/database.js

Explanation: This layered approach makes it easy to change database schemas without affecting controllers, or change business logic without affecting routes. Each layer has a clear responsibility.

Relational Database: MySQL 8.0

Yearly Database System: Separate database per year (masjid_app_2024, masjid_app_2025)

Master Database: masjid_master (tracks active years)

Schema Design: Normalized database structure

Main Tables:

users - Main user table (IC as primary key)

students - Student-specific data (references users.ic)

teachers - Teacher-specific data (references users.ic)

classes - Class information

attendance - Attendance records

exams - Exam information

results - Exam results

fees - Fee records and payments

user_roles - User role assignments (supports multiple roles)

```
CREATE TABLE users (  
  ic VARCHAR(20) PRIMARY KEY,  
  nama VARCHAR(100) NOT NULL,  
  umur INT,  
  alamat VARCHAR(255),  
  telepon VARCHAR(20),  
  email VARCHAR(100) UNIQUE,  
  password VARCHAR(255),  
  role ENUM('student','teacher','admin') NOT NULL,  
  status ENUM('aktif','tidak_aktif','cuti') DEFAULT 'aktif',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE students (  

```

```
user_ic VARCHAR(20) PRIMARY KEY,  
kelas_id INT,  
tarikh_daftar DATE,  
FOREIGN KEY (user_ic) REFERENCES users(ic) ON DELETE CASCADE,  
FOREIGN KEY (kelas_id) REFERENCES classes(id) ON DELETE SET NULL  
);
```

Example Schema:

Recommended Patterns:

Module Pattern: Organize code into modules

Service Pattern: Separate business logic into services

Repository Pattern: Abstract database access (if applicable)

Middleware Pattern: Express middleware for cross-cutting concerns

Factory Pattern: Create objects with flexible creation logic

Singleton Pattern: Database connections, configuration

Example from MyMasjidApp:

```
// backend/services/userRoleService.js  
export const fetchUserRoles = async (userIc, primaryRole) => {  
  // Fetch all roles for a user from user_roles table  
  // Return array of role strings  
};
```

```
// backend/services/studentService.js  
export const fetchStudentByIc = async (ic) => {  
  // Fetch student data by IC number  
  // Return student object  
};
```

Service Pattern (backend/services/):

```
// backend/controllers/studentController.js  
export const getAllStudents = async (req, res) => {  
  try {  
    const { page = 1, limit = 50, search } = req.query;  
    // Fetch students with pagination and search  
    // Return formatted response  
  } catch (error) {  
    // Error handling  
  }  
};
```



```
}  
};
```

Controller Pattern (backend/controllers/):

```
// backend/routes/students.js  
import { getAllStudents, createStudent } from '../controllers/studentController.js';  
import { authenticateToken, requireRole } from '../middleware/auth.js';  
  
router.route('/')  
  .get(authenticateToken, getAllStudents)  
  .post(authenticateToken, requireRole(['admin']), createStudent);
```

Route Pattern (backend/routes/):

Organize code into logical modules

Keep modules focused and cohesive

Minimize coupling between modules

Use clear module interfaces

Design for horizontal scaling

Use stateless design where possible

Implement caching strategies

Optimize database queries

Consider microservices for large features

Write self-documenting code

Keep functions small and focused

Use meaningful names

Document complex logic

Follow consistent patterns

-

Test individual functions and methods in isolation

Mock external dependencies

Aim for high code coverage (target: 80%+)

Run quickly and frequently

Tools/Frameworks:

Jest (JavaScript testing framework)

React Testing Library (for React components)

Test interactions between components - Integration tests verify that different parts of the application work together correctly

Test API endpoints with database - Tests make real API calls and interact with a test database, ensuring the full request-response cycle works

Test service layer integrations - Services are tested together with their dependencies (like database queries)

Verify data flow through system - Integration tests ensure data flows correctly through layers (route → controller → service → database)

Explanation: While unit tests verify individual functions work, integration tests verify that different parts work together. For example, an integration test might create a student via the API and verify the student appears in the database.

Tools/Frameworks:

Jest with Supertest (for API testing)

Test database setup

Test complete user workflows

Test end-to-end scenarios

Test system under realistic conditions

Verify system meets requirements

Tools/Frameworks:

Manual testing procedures

Automated E2E testing (if implemented)

Verify features meet acceptance criteria

Test from user perspective

Validate business requirements

User acceptance testing (UAT)

Primary Tools:

Jest: JavaScript testing framework

React Testing Library: React component testing

Supertest: HTTP assertion library for API testing

```
tests/
├── unit/
│   ├── services/
│   │   ├── userRoleService.test.js
│   │   └── studentService.test.js
│   ├── utils/
│   │   ├── icUtils.test.js
│   │   └── passwordPolicy.test.js
│   └── components/
│       ├── Login.test.jsx
│       └── Pelajar.test.jsx
├── integration/
│   ├── api/
│   │   ├── auth.test.js
│   │   └── students.test.js
│   ├── database/
│   │   └── queries.test.js
└── e2e/
    └── user-flow.test.js
```

Test Structure (Recommended):

Note: Currently, MyMasjidApp uses manual testing procedures. Automated testing frameworks can be added as the project grows.

Bug Report Template:

Title: Clear, concise description

Severity: Critical, High, Medium, Low

Priority: P1, P2, P3, P4

Environment: Browser, OS, version

Steps to Reproduce: Detailed steps

Expected Behavior: What should happen

Actual Behavior: What actually happens

Screenshots/Logs: Visual evidence

Additional Context: Any other relevant information

Bug Workflow:

Bug reported with template

Bug triaged and assigned priority

Bug assigned to developer

Developer fixes bug

Fix verified through testing

Bug marked as resolved

Bug closed after verification

Target Metrics:

Code Coverage: Minimum 70%, Target 80%+

Code Quality: Maintain A rating (if using tools like SonarQube)

Test Coverage: All critical paths covered

Performance: Response time < 500ms for API calls

Tools:

Jest coverage reports

ESLint for code quality

Code review process

-

Inline Comments:

Explain complex logic

Document non-obvious behavior

Include TODO/FIXME notes with context

Reference related issues/tickets

Function Documentation:

Use JSDoc format

Document parameters and return types

Include examples for complex functions

Document exceptions/thrown errors

Architecture Documentation:

System architecture diagrams

Database schema diagrams

API endpoint documentation

Data flow diagrams

Design Decisions:

Document major design decisions

Explain trade-offs considered

Record alternatives evaluated

User Documentation:

User Manual (see USER_GUIDE.md)

Feature-specific guides

Troubleshooting guides

Technical Documentation:

API documentation

Deployment guides

Development setup guides

Configuration guides

Markdown (.md): Primary documentation format

JSDoc: Code documentation

README.md: Project overview and quick start

Word Documents (.docx): User manuals (generated from Markdown)

Diagrams: Mermaid syntax in Markdown files

MyMasjidApp/

```
|— README.md          # Project overview
|— USER_GUIDE.md      # End-user guide
|— DEVELOPMENT_STANDARDS.md  # This file
|— DEPLOYMENT_GUIDE.md  # Deployment instructions
|— SECURITY_ARCHITECTURE.md # Security documentation
|— SYSTEM_WORKFLOWS.md  # Workflow documentation
|— TECH_STACK.md        # Technology stack details
|— SCREENSHOT_LINKS.md  # Screenshot references
|— database/
|   |— masjid_app_schema.sql # Database schema
|   |— .cursor/
|       |— rules/
|           |— auto-deploy.mdc # Deployment automation rules
```

Documentation Structure (MyMasjidApp):

Word Documents (Generated):

MyMasjidApp_User_Manual_[timestamp].docx - User manual

MyMasjidApp_Admin_Manual_[timestamp].docx - Admin manual

MyMasjidApp_Development_Standards_[timestamp].docx - Development standards

•

npm run build

Frontend Build:

Production build with Vite

Optimized and minified assets

Environment variables injected

Backend Build:

Docker container build

No separate build step (Node.js runtime)

Automated Builds:

Manual builds currently (CI/CD can be added)

Docker Compose for local testing

Production builds on deployment server

Development:

Local development on developer machines - Developers run the application on their own computers

Uses Docker Compose - Docker Compose runs all services (database, backend, frontend) together for local development

Hot reload enabled - Code changes are automatically reflected without restarting the server, speeding up development

Debug mode enabled - Detailed error messages and logging help developers debug issues

Why separate environments: Development environments allow developers to test changes without affecting production users.

Staging:

Production-like environment for testing - Staging mirrors production but uses test data

Uses production-like configuration - Testing in a production-like environment catches issues that don't appear in development

Testing of new features before production - Features are tested in staging before being released to users

Optional environment - Staging is recommended but not required for smaller projects

Explanation: Staging environments reduce the risk of production bugs by catching issues before they reach users.

Production:

Live application environment - This is where real users access the application

Uses production configuration - Production settings prioritize security and performance

SSL/TLS enabled - HTTPS encrypts data in transit, protecting user information

Monitoring and logging enabled - Production monitoring helps detect and fix issues quickly

Why this matters: Production environments require careful configuration to ensure security, reliability, and performance for real users.

Code Review: Ensure code is reviewed and approved

Build: Build frontend and backend containers

Test: Run tests in staging (if available)

Backup: Backup production database

Deploy: Deploy using Docker Compose

Verify: Verify deployment success

Monitor: Monitor application health

Rollback: Prepare rollback plan if needed

Frontend deployment

npm run build

docker-compose build frontend

docker-compose up -d frontend

Backend deployment

docker-compose build backend

docker-compose up -d backend

Full deployment (as per auto-deploy.mdc rule)

npm run build && docker-compose build frontend && docker-compose build backend && docker-compose up -d frontend && docker-compose up -d backend

Verify deployment

docker-compose ps

docker-compose logs backend --tail=50

docker-compose logs frontend --tail=50

Database migration (if needed)

docker-compose exec backend npm run migrate

Deployment Commands (MyMasjidApp):

Auto-Deployment Rule:

After ANY code changes, deployment is MANDATORY

Frontend changes require: npm run build → docker-compose build frontend → docker-compose up -d frontend

Backend changes require: docker-compose build backend → docker-compose up -d backend

Always rebuild Docker images (don't just restart) to load new code

Semantic Versioning (SemVer):

Format: MAJOR.MINOR.PATCH

MAJOR: Breaking changes

MINOR: New features (backward compatible)

PATCH: Bug fixes (backward compatible)

Examples:

1.0.0: Initial release

1.1.0: Added new feature

1.1.1: Fixed bug

2.0.0: Breaking changes

Release Numbering:

Tag releases in Git: v1.0.0

Document release notes

Update version in package.json

Rollback Procedures:

Identify issue in production

Assess severity and impact

Revert code changes (Git revert)

Rebuild and redeploy previous version

Restore database backup if needed

Verify rollback success

Document incident

Prevention:

Test thoroughly before deployment

Use staging environment

Gradual rollout when possible

Monitor after deployment

Keep backups available

-

Authentication:

JWT tokens for session management

Password hashing using bcrypt (minimum 10 rounds)

Token expiration and refresh mechanisms

Secure password requirements (minimum 6 characters, complexity recommended)

Authorization:

Role-based access control (RBAC) - Users have roles (student, teacher, admin) that determine what actions they can perform

Permission checks on API endpoints - Backend routes use middleware (like `requireRole(['admin'])`) to verify users have the necessary permissions

Frontend route protection - React Router's `ProtectedRoute` component prevents unauthorized users from accessing certain pages

Middleware for authorization checks - Middleware functions check user permissions before allowing requests to reach controllers

Explanation: Authorization ensures users can only access resources and perform actions they are allowed to. For example, only admins can create students, and students can only view their own data. This multi-layer approach (frontend and backend) provides defense in depth.

Best Practices:

Never store passwords in plain text

Use HTTPS for all communications

Implement token refresh mechanism

Log authentication attempts

Implement account lockout after failed attempts

Data at Rest:

Database encryption (MySQL encryption)

File system encryption for sensitive files

Backup encryption

Data in Transit:

HTTPS/TLS for all communications

SSL certificates (Let's Encrypt)

Secure API endpoints

Sensitive Data:

Encrypt sensitive user data

Hash passwords with bcrypt

Secure API keys and secrets

Use environment variables for configuration

Do NOT:

Commit secrets, passwords, or API keys to Git

Log sensitive information

Expose sensitive data in error messages

Store credentials in code

Do:

Use environment variables for secrets

Use .env files (not committed to Git)

Rotate keys regularly

Use secure password storage (bcrypt hashing)

Encrypt sensitive data (IC numbers, phone numbers, addresses)

Database

DB_HOST=mysql

DB_PORT=3306

DB_USER=root

DB_PASSWORD=masjid_password

DB_NAME=masjid_app

JWT

JWT_SECRET=your_jwt_secret_here

JWT_EXPIRES_IN=24h

Server

PORT=5000

NODE_ENV=production

FRONTEND_URL=http://localhost:3000

Email

EMAIL_USER=your_email@example.com

EMAIL_PASSWORD=your_app_password

SMS (Twilio - optional)

TWILIO_ACCOUNT_SID=

TWILIO_AUTH_TOKEN=

TWILIO_PHONE_NUMBER=

Payment Gateway (ToyyibPay)

TOYYIBPAY_SECRET_KEY=your_secret_key

TOYYIBPAY_CATEGORY_CODE=your_category_code

Environment Variables (backend/env.example):

Note: Never commit .env files. Use env.example as a template.

OWASP Top 10 Considerations:

Injection: Use parameterized queries, input validation

Broken Authentication: Secure authentication mechanisms

Sensitive Data Exposure: Encrypt sensitive data

XML External Entities (XXE): Not applicable (no XML processing)

Broken Access Control: Implement proper authorization

Security Misconfiguration: Secure default configurations

XSS (Cross-Site Scripting): Sanitize user input, use React's built-in protection

Insecure Deserialization: Validate and sanitize input

Using Components with Known Vulnerabilities: Keep dependencies updated

Insufficient Logging & Monitoring: Implement proper logging

Input Validation:

Validate all user input

Sanitize input before processing

Use whitelist validation where possible

Validate data types and formats

SQL Injection Prevention:

Use parameterized queries (mysql2 prepared statements)

Never concatenate user input into SQL

Always use pool.execute() with parameter placeholders

```
// ✅ SAFE - Uses prepared statement
const [users] = await pool.execute(
  "SELECT FROM users WHERE REPLACE(REPLACE(ic, '-', ''), ' ', '') = ?",
  [normalizedIC]
);
```

```
// ✅ SAFE - Parameterized query
await pool.execute(
  'INSERT INTO users (ic, nama, email, password, role) VALUES (?, ?, ?, ?, ?)',
  [ic, nama, email, hashedPassword, role]
);
```

```
// ❌ UNSAFE - SQL injection risk
await pool.execute(`SELECT FROM users WHERE ic = '${userIc}'`);
```

Example from MyMasjidApp (backend/controllers/authController.js):

XSS Prevention:

React escapes content by default

Sanitize user-generated content

Use Content Security Policy (CSP)

-

Bug Severity Levels:

Critical: System down, data loss, security breach

High: Major feature broken, significant impact

Medium: Feature partially working, workaround available

Low: Minor issue, cosmetic problem

Bug Priority:

P1: Fix immediately (Critical)

P2: Fix within 24 hours (High)

P3: Fix within 1 week (Medium)

P4: Fix when time permits (Low)

Bug Lifecycle:

Reported

Triaged

Assigned

In Progress

Testing

Resolved

Closed

Security Patches:

Apply security patches immediately

Test patches in staging first

Monitor after deployment

Document patch application

Dependency Updates:

Review dependency updates regularly

Update minor/patch versions monthly

Major version updates require testing

Keep changelog of updates

Version Management:

Maintain changelog

Document patches and updates

Version tagging in Git

Release notes for users

Deprecation Process:

Announce deprecation (30-90 days notice)

Document migration path

Provide support during transition

Set end-of-life date

Remove deprecated features

Support Levels:

Active Support: Full support and updates

Maintenance Support: Security patches only

End of Life: No support

-

Recommended IDEs:

VS Code: Recommended editor with extensions

WebStorm: Alternative IDE option

Cursor: AI-assisted coding (as used in project)

Required Extensions/Tools:

ESLint extension

Prettier extension

Git integration

Docker extension

MySQL client tools

Libraries and Frameworks:

React 19.1.1

Express.js

MySQL 8.0

Docker & Docker Compose

Vite

TailwindCSS

Axios

Jest: JavaScript testing framework

React Testing Library: Component testing

Supertest: API testing

Postman/Insomnia: API testing tools (optional)

Communication:

Email for formal communications

Project repository issues for tracking

Documentation in Markdown files

Project Management:

Git issues for task tracking

GitHub/GitLab projects (if using)

Documentation for requirements

Development Tools:

Git for version control

Docker for containerization

npm for package management

MySQL Workbench / DBeaver for database

-

Software Developer:

Write code following standards

Write unit tests

Participate in code reviews

Update documentation

Fix bugs and issues

Senior Developer / Technical Lead:

Review code and provide guidance

Make architectural decisions

Mentor junior developers

Ensure code quality

Coordinate technical work

Quality Assurance Engineer:

Write and execute test cases

Report bugs

Verify bug fixes

Test new features

Maintain test documentation

DevOps Engineer:

Manage deployment infrastructure

Configure CI/CD pipelines

Monitor system health

Manage server environments

Handle deployment processes

Project Manager:

Coordinate development activities

Manage requirements and backlog

Track progress

Facilitate communication

Plan releases

All Team Members:

Follow coding standards

Participate in code reviews

Update documentation

Communicate issues and blockers

Test their own work

Developers:

Write clean, maintainable code

Write tests for new features

Fix bugs in a timely manner

Seek help when needed

Share knowledge with team

-

License Types:

Review licenses of all dependencies

Prefer MIT, Apache 2.0, BSD licenses

Avoid GPL unless appropriate

Document licenses in project

License Compliance:

Include license files

Document third-party licenses

Comply with license requirements

Review licenses before adding dependencies

Current Licenses:

Most dependencies use MIT/Apache licenses

Review package.json for license information

Check LICENSE files in dependencies

General Data Protection:

Follow data protection best practices

Implement user data privacy controls

Provide data access and deletion capabilities

Secure user data storage

Regulations to Consider:

GDPR (if serving EU users): Right to access, deletion, portability

PDPA (Malaysia): Personal Data Protection Act

Local data protection laws

Data Handling:

Collect only necessary data

Inform users about data collection

Secure data storage and transmission

Provide data deletion options

Document data processing activities

Code Ownership:

Project code belongs to the organization

Developers contribute code under project ownership

Third-party code must be properly licensed

Open Source Contributions:

Review before contributing to open source

Follow organization policies

Ensure no conflicts with project code

Documentation:

Document IP considerations

Maintain license information

Track third-party code usage

-

This section provides complete, ready-to-use templates for common development tasks. Copy these templates and customize them as needed for your specific use case.

Use this checklist when reviewing code changes to ensure quality and consistency:

- ☐ Code follows naming conventions

Variables use camelCase (e.g., userName, isActive)

Functions use camelCase with verb-noun pattern (e.g., getUserById, createStudent)

Constants use UPPER_SNAKE_CASE (e.g., MAX_RETRY_COUNT, SESSION_DURATION_SECONDS)

React components use PascalCase (e.g., Login, PelajarList)

File names follow conventions (camelCase for JS, PascalCase for React components)

- ☐ Code is properly formatted

Uses 2 spaces for indentation (no tabs)

Lines are within 100 character limit

Consistent formatting throughout the file

Proper spacing and alignment

- ☐ Functions have documentation

Public functions have JSDoc comments

Parameters and return types are documented

Complex functions have usage examples

Comments explain "why", not "what"

- ☐ Error handling is implemented

Try-catch blocks for async operations

Meaningful error messages (user-friendly in Malay for MyMasjidApp)

Errors are logged appropriately

No sensitive information in error messages

Graceful error handling (no unhandled promise rejections)

- ☐ Tests are included and passing

Unit tests for new functions

Integration tests for API endpoints (if applicable)

All tests pass

Test coverage is adequate

- ☐ No security vulnerabilities

Uses parameterized queries (no SQL injection)

Input validation and sanitization

No hardcoded secrets or passwords

Authentication/authorization checks present

No XSS vulnerabilities (React escapes by default)

- ☐ Performance is acceptable

Database queries are optimized (no N+1 queries)

No unnecessary re-renders (React optimization)

Efficient algorithms and data structures

Caching used where appropriate

No blocking operations in main thread

- ☐ Documentation is updated

README updated if needed

API documentation updated

Code comments are clear and current

User-facing documentation updated (if applicable)

- ☐ No console.log statements in production code

Console.log removed or replaced with proper logging

Debug statements removed

Only necessary logging remains (errors, important events)

- ☐ No commented-out code

Dead code removed

Commented code removed (Git history preserves it)

Only active, working code remains

Use this checklist before submitting code to ensure thorough testing:

- ☐ Unit tests written for new functions

Each new function has corresponding unit tests

Tests cover normal operation (happy path)

Tests are isolated and can run independently

External dependencies are mocked

- ☐ Integration tests for API endpoints

API endpoints are tested with real database connection (test database)

Request/response cycle is verified

Authentication and authorization are tested

Error responses are verified

- ☐ Manual testing completed

Feature works as expected in the UI

All user interactions work correctly

Navigation and routing work properly

Forms validate and submit correctly

- ☐ Edge cases tested

Empty inputs handled correctly

Null/undefined values handled

Boundary values tested (min/max limits)

Invalid data formats rejected properly

- ☐ Error scenarios tested

Network errors handled gracefully

Server errors display appropriate messages

Validation errors show correctly

Authentication errors redirect properly

- ☐ Performance tested

Page load times are acceptable (< 3 seconds)

API responses are fast (< 500ms)

No memory leaks (check with browser DevTools)

Database queries are optimized

- ☐ Security tested

Authentication required for protected routes

Authorization checks prevent unauthorized access

Input validation prevents injection attacks

Sensitive data is not exposed in responses

CSRF protection verified (if applicable)

- ☐ Browser compatibility tested

Works in Chrome (latest version)

Works in Firefox (latest version)

Works in Safari (latest version)

Works in Edge (latest version)

Fallbacks for unsupported features

- [] Mobile responsiveness tested (if applicable)

Layout adapts to mobile screen sizes

Touch interactions work correctly

Forms are usable on mobile

Navigation is accessible on small screens

Use this template when reporting bugs to ensure all necessary information is included:

Title: [Clear, concise description of the bug]

Example: "Login fails with 'Invalid credentials' error for valid users"

Keep it short but descriptive (one sentence)

Severity: [Critical/High/Medium/Low]

Critical: System down, data loss, security breach - Fix immediately

High: Major feature broken, significant impact - Fix within 24 hours

Medium: Feature partially working, workaround available - Fix within 1 week

Low: Minor issue, cosmetic problem - Fix when time permits

Priority: [P1/P2/P3/P4]

P1: Fix immediately (Critical severity)

P2: Fix within 24 hours (High severity)

P3: Fix within 1 week (Medium severity)

P4: Fix when time permits (Low severity)

Environment:

Browser: [Browser name and version] (e.g., Chrome 120.0.6099.129, Firefox 121.0)

OS: [Operating system] (e.g., Windows 11, macOS 14.2, Ubuntu 22.04)

Device: [Desktop/Mobile/Tablet] (e.g., Desktop, iPhone 14, iPad Pro)

User Role: [Student/Teacher/Admin/IB/PIC] (if applicable)

URL/Route: [Specific page or API endpoint where bug occurs]

Steps to Reproduce:

- [Step 1 - What was the first action taken?]
- [Step 2 - What happened next?]
- [Step 3 - What was the last action before the bug appeared?]
- [Continue with additional steps if needed]

Example:

Navigate to /login page

Enter valid IC number: 051003060229

Enter valid password: 123456

Click "Log Masuk" button

Error message appears: "Invalid credentials"

Expected Behavior: [What should happen when following the steps]

Example: "User should be successfully logged in and redirected to dashboard"

Actual Behavior: [What actually happens]

Example: "Error message 'Invalid credentials' appears, user remains on login page"

Screenshots/Logs: [Attach screenshots or error logs]

Screenshot of the error (if visual bug)

Browser console errors (F12 → Console)

Network tab errors (F12 → Network)

Backend logs (if available)

Error message text (copy exact text)

Additional Context: [Any other relevant information]

Does this happen consistently or intermittently?

When did this first occur?

Did it work before? If so, what changed?

Any error codes or IDs?

Related issues or pull requests?

Workaround (if any)

-

Use this template when creating pull requests to ensure all necessary information is included:

Title: [Type(Scope)]: Brief description

Example: feat(auth): add password reset functionality

Example: fix(students): resolve IC validation error

Example: docs(api): update authentication endpoints documentation

Description:

What: [What changes were made? Describe the changes clearly]

Why: [Why were these changes needed? What problem do they solve?]

How: [How do these changes work? Briefly explain the implementation]

Impact: [What is the impact of these changes? Who is affected?]

Type of Change:

- ☐ Bug fix (non-breaking change which fixes an issue)
- ☐ New feature (non-breaking change which adds functionality)
- ☐ Breaking change (fix or feature that would cause existing functionality to not work as expected)
- ☐ Documentation update
- ☐ Code refactoring (no functional changes)
- ☐ Performance improvement
- ☐ Security fix

Related Issues:

Closes #[issue-number]

Fixes #[issue-number]

Related to #[issue-number]

Changes Made:

- [List of changes made]

Example:

Added password reset API endpoint (POST /api/auth/password-reset)

Added email service integration for sending reset codes

Added password reset form component

Updated authentication controller to handle reset requests

Testing:

- ☐ Unit tests added/updated
- ☐ Integration tests added/updated
- ☐ Manual testing completed
- ☐ Tested in development environment
- ☐ Tested in staging environment (if applicable)

How to Test:

- [Step 1 - How to test the changes]
- [Step 2 - What to verify]
- [Step 3 - Expected results]

Example:

Navigate to /login page

Click "Lupa Kata Laluan?" link

Enter IC number and email

Verify reset code is received via email

Enter reset code and new password

Verify password is changed and user can login

Screenshots (if applicable): [Attach screenshots of UI changes, before/after comparisons, etc.]

Checklist:

- ☐ Code follows project coding standards
- ☐ Code is properly formatted
- ☐ Functions have documentation (JSDoc comments)
- ☐ Error handling is implemented
- ☐ Tests are included and passing
- ☐ No security vulnerabilities
- ☐ Performance is acceptable
- ☐ Documentation is updated (README, API docs, etc.)

- [] No console.log statements in production code
- [] No commented-out code
- [] Reviewed my own code before submitting
-

Use this template for commit messages following Conventional Commits format:

<type>(<scope>): <subject>

<body>

<footer>

Types:

feat: New feature

fix: Bug fix

docs: Documentation changes

style: Code style changes (formatting, semicolons, etc.)

refactor: Code refactoring (no functional changes)

test: Test additions/changes

chore: Maintenance tasks (dependencies, build, etc.)

perf: Performance improvements

ci: CI/CD changes

build: Build system changes

Scopes:

auth: Authentication/authorization

students: Student management

teachers: Teacher management

classes: Class management

attendance: Attendance tracking

fees: Fee management

exams: Exam management

results: Results management

api: API changes

ui: UI/components

db: Database changes

config: Configuration changes

security: Security changes

Examples:

feat(auth): add password reset functionality

Implement password reset flow with email verification.
Users can request password reset via IC number and email.
Reset codes are sent via email service.
Password can be reset using reset code.

Closes #123

Feature:

fix(students): resolve IC validation error for hyphens

Fixed IC validation to accept formats with and without hyphens.
Normalization function now handles both formats correctly.
Updated validation middleware to use normalized format.

Fixes #456

Bug Fix:

docs(api): update authentication endpoints documentation

Added examples for login, register, and password reset endpoints.
Included request/response formats and error codes.
Updated API documentation with new endpoints.

Related to #789

Documentation:

refactor(students): extract IC normalization to utility function

Moved IC normalization logic from controller to utility function.

Created normalizeICForQuery function in utils/icUtils.js.
Updated all controllers to use new utility function.

Improves code reusability and maintainability.

Refactoring:

-

Use this template when requesting new features:

Feature Title: [Clear, concise feature name]

Example: "Monthly Fee Auto-Generation"

Example: "Student Attendance SMS Notifications"

Description: [Detailed description of the feature. What should it do? What problem does it solve?]

Example: The system should automatically generate monthly fees for all active students on the first day of each month. This will eliminate the need for manual fee entry and ensure all students are billed correctly.

Use Case: [Who will use this feature? What scenario will it support?]

Example: Admin users will use this feature to automatically generate fees for all students each month, reducing manual work and ensuring consistency.

User Story: As a [user type], I want to [action] so that [benefit].

Example: As an admin, I want the system to automatically generate monthly fees so that I don't have to manually create fees for each student.

Requirements:

- [Requirement 1]
- [Requirement 2]
- [Requirement 3]

Example:

System should generate fees on the 1st of each month

Fees should be generated for all active students

Admin should be able to configure fee amounts per class

Admin should be notified when fees are generated

Admin should be able to manually trigger fee generation if needed

Acceptance Criteria:

- ☐ [Criterion 1 - What must be true for this feature to be considered complete?]
- ☐ [Criterion 2]
- ☐ [Criterion 3]

Example:

- ☐ Fees are automatically generated on the 1st of each month at 12:00 AM
- ☐ All active students receive fee records
- ☐ Admin receives email notification with generation summary
- ☐ Admin can view generated fees in the fee management page
- ☐ Admin can manually trigger fee generation from admin dashboard

Mockups/Wireframes (if applicable): [Attach mockups, wireframes, or screenshots showing the expected UI]

Priority:

- ☐ Critical - Blocks other work
- ☐ High - Important feature, should be implemented soon
- ☐ Medium - Nice to have, can wait
- ☐ Low - Future consideration

Related Issues:

Related to #[issue-number]

Depends on #[issue-number]

Blocks #[issue-number]

Additional Notes: [Any other relevant information, constraints, dependencies, etc.]

-

Use this template when requesting changes to existing functionality:

Change Title: [Clear, concise change name]

Example: "Modify Fee Payment Flow to Support Partial Payments"

Example: "Update Student Registration to Require Email"

Description: [What needs to be changed? What is the current behavior? What should the new behavior be?]

Current Behavior: [How does the system currently work?]

Example: Currently, students can only pay fees in full. If a student wants to pay RM 50 of a RM 150 fee, they must pay the full amount.

Proposed Behavior: [How should the system work after the change?]

Example: Students should be able to pay any amount up to the full fee amount. The system should track partial payments and show remaining balance.

Reason for Change: [Why is this change needed? What problem does it solve?]

Example: Students often cannot afford to pay full fees at once. Allowing partial payments will make the system more flexible and user-friendly.

Impact Analysis:

Affected Components: [List components/features that will be affected]

Fee payment flow

Fee status calculation

Fee reports and outstanding balance display

Breaking Changes: [Will this break existing functionality?]

No breaking changes expected

Backward compatible with existing full payment flow

Migration Required: [Will data migration be needed?]

No data migration required

Existing fee records remain unchanged

Requirements:

- [Requirement 1]
- [Requirement 2]

Example:

Payment form should allow entering custom amount (up to full fee)

Fee status should update to "partial" when amount paid is less than total

Outstanding balance should be calculated and displayed

Payment history should show partial payment amounts

Acceptance Criteria:

- ☐ [Criterion 1]
- ☐ [Criterion 2]

Example:

- ☐ Students can enter custom payment amount in payment form
- ☐ System accepts payment amounts less than full fee
- ☐ Fee status updates to "partial" when applicable
- ☐ Outstanding balance is calculated correctly
- ☐ Payment history shows partial payment details

Priority:

- ☐ Critical - Blocks other work
- ☐ High - Important change, should be implemented soon
- ☐ Medium - Nice to have, can wait
- ☐ Low - Future consideration

Related Issues:

Related to #[issue-number]

Depends on #[issue-number]

Additional Notes: [Any other relevant information, constraints, dependencies, etc.]

-

Use this template when leaving code review comments:

Type: [Suggestion/Question/Blocking Issue/Must Fix/Nice to Have]

Location: [File path and line number]

Example: backend/controllers/authController.js:45

Comment: [Your comment about the code]

Example: Consider using a constant for the session duration instead of hardcoding the value. This makes it easier to change later if needed.

// Instead of:

```
const token = jwt.sign(payload, secret, { expiresIn: '24h' });
```

// Consider:

```
const SESSION_DURATION = '24h';
```

```
const token = jwt.sign(payload, secret, { expiresIn: SESSION_DURATION });
```

Suggestion: [If applicable, suggest how to fix or improve]

Reference: [If applicable, link to coding standards, documentation, or related issues]

See DEVELOPMENT_STANDARDS.md section 3.3 (Constants)

-

Use this template when creating release notes:

Version: [Version number]

Example: 1.2.0

Release Date: [Date]

Example: January 15, 2025

Release Type:

- ☐ Major Release (breaking changes)
- ☐ Minor Release (new features, backward compatible)
- ☐ Patch Release (bug fixes)

Summary: [Brief summary of the release - one or two sentences]

Example: This release adds password reset functionality, improves fee management with partial payment support, and fixes several critical bugs in the attendance tracking system.

New Features:

- [Feature 1 with description]
- [Feature 2 with description]

Example:

Password Reset: Users can now reset their passwords via email or SMS. Reset codes are sent to registered email or phone number.

Partial Fee Payments: Students can now make partial payments for fees. The system tracks outstanding balances and updates fee status accordingly.

Enhanced Attendance Reports: New attendance report features include monthly summaries, class-level statistics, and export functionality.

Improvements:

- [Improvement 1 with description]
- [Improvement 2 with description]

Example:

Improved performance of student search functionality (50% faster)

Enhanced UI responsiveness on mobile devices

Better error messages for validation failures

Bug Fixes:

- [Bug fix 1 with description]
- [Bug fix 2 with description]

Example:

Fixed IC validation error that rejected valid IC numbers with hyphens

Resolved issue where attendance records were not saving correctly

Fixed fee calculation error for students with multiple classes

Security:

- [Security fix or enhancement 1]
- [Security fix or enhancement 2]

Example:

Enhanced password hashing security (increased bcrypt rounds)

Fixed XSS vulnerability in announcement display

Improved SQL injection protection in search queries

Breaking Changes:

- [Breaking change 1 with migration instructions]
- [Breaking change 2 with migration instructions]

Example:

API endpoint `/api/students/search` now requires authentication (previously public)

Database schema changes require running migration script: `npm run migrate`

Deprecations:

- [Deprecated feature 1 with removal date and migration path]
- [Deprecated feature 2 with removal date and migration path]

Example:

Deprecated `/api/students/old-search` endpoint. Will be removed in version 2.0.0. Please use `/api/students/search` instead.

Upgrade Instructions: [Step-by-step upgrade instructions if needed]

Example:

Backup database

Pull latest code: `git pull origin main`

Install dependencies: `npm install`

Run migrations: `npm run migrate`

Rebuild containers: `docker-compose build`

Restart services: `docker-compose up -d`

Known Issues:

- [Known issue 1]
- [Known issue 2]

Example:

Fee generation may take longer than usual for large student databases (work in progress)

Mobile UI may display incorrectly on iOS Safari versions below 15.0

Credits: [Thank contributors, acknowledge help, etc.]

Example: Thanks to the development team for their hard work on this release. Special thanks to [Name] for the password reset feature and [Name] for the partial payment implementation.

-

Use this template when documenting API endpoints:

Endpoint: [HTTP Method] [Endpoint Path]

Example: `POST /api/auth/login`

Example: `GET /api/students/:ic`

Description: [What does this endpoint do?]

Example: Authenticates a user with IC number and password. Returns a JWT token and user information upon successful authentication.

Authentication:

- [] Required
- [] Optional
- [] Not required

Content-Type: application/json

Authorization: Bearer {token} (if authentication required)

Required Headers:

Request Parameters:

Path Parameters:

parameter-name (type) - [Description]

Example: ic (string) - Student IC number (12 digits, with or without hyphens)

Query Parameters:

parameter-name (type, optional) - [Description]

Example: page (integer, optional) - Page number for pagination (default: 1)

Example: limit (integer, optional) - Number of items per page (default: 50, max: 1000)

```
{  
  "field1": "value1",  
  "field2": "value2"  
}
```

Request Body:

```
{  
  "icNumber": "051003060229",  
  "password": "123456"  
}
```

Example:

Request Body Schema:

field-name (type, required/optional) - [Description]

Example: icNumber (string, required) - User IC number (12 digits)

Example: password (string, required) - User password (minimum 6 characters)

Success Response:

Status Code: [HTTP status code]

Example: 200 OK

Example: 201 Created

```
{
  "success": true,
  "message": "Success message",
  "data": {
    "field1": "value1",
    "field2": "value2"
  }
}
```

Response Body:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "ic": "051003060229",
      "nama": "Ahmad Zulkifli",
      "email": "ahmad@example.com",
      "role": "student"
    }
  }
}
```

Example:

Error Responses:

Status Code: [HTTP status code]

Example: 400 Bad Request

Example: 401 Unauthorized

Example: 404 Not Found

Example: 500 Internal Server Error

```
{
  "success": false,
  "message": "Error message",
}
```

```
"errors": [  
  {  
    "field": "field-name",  
    "message": "Error message for this field"  
  }  
]  
}
```

Response Body:

```
{  
  "success": false,  
  "message": "Invalid credentials",  
  "errors": []  
}
```

Example:

```
{  
  "success": false,  
  "message": "Validation failed",  
  "errors": [  
    {  
      "field": "icNumber",  
      "message": "IC number must be exactly 12 digits"  
    },  
    {  
      "field": "password",  
      "message": "Password must be at least 6 characters"  
    }  
  ]  
}
```

Example with Validation Errors:

Example Requests:

```
curl -X POST http://localhost:5000/api/auth/login \  
-H "Content-Type: application/json" \  
-d '{  
  "icNumber": "051003060229",  
  "password": "123456"  
}'
```

cURL:

```
const response = await axios.post('/api/auth/login', {  
  icNumber: '051003060229',  
  password: '123456'  
});
```

JavaScript (Axios):

Notes: [Any additional notes, warnings, limitations, etc.]

Example:

Token expires after 24 hours. Use refresh token to obtain new access token.

Rate limiting: 5 requests per 15 minutes per IP address.

Password is hashed using bcrypt before storage.

Related Endpoints:

- [Related endpoint 1]
- [Related endpoint 2]

Example:

POST /api/auth/register - Register new user

POST /api/auth/password-reset - Reset password

GET /api/auth/profile - Get user profile

Industry Standards:

ISO/IEC 25010: Software Quality Model

Defines quality characteristics for software systems

Covers functional suitability, performance, compatibility, usability, reliability, security, maintainability, and portability

Reference: <https://www.iso.org/standard/35733.html>

OWASP Top 10: Web Application Security Risks

Lists the most critical web application security risks

Includes injection, broken authentication, sensitive data exposure, XML external entities (XXE), broken access control, security misconfiguration, XSS, insecure deserialization, using components with known vulnerabilities, and insufficient logging

Reference: <https://owasp.org/www-project-top-ten/>

Semantic Versioning (SemVer): Version numbering (semver.org)

Version format: MAJOR.MINOR.PATCH (e.g., 1.2.3)

MAJOR: Breaking changes, MINOR: New features (backward compatible), PATCH: Bug fixes (backward compatible)

Reference: <https://semver.org/>

Conventional Commits: Commit message format

Format: <type>(<scope>): <subject>

Types: feat, fix, docs, style, refactor, test, chore, perf, ci

Makes commit history more readable and enables automated tools

Reference: <https://www.conventionalcommits.org/>

Best Practices:

SOLID Principles: Object-oriented design

Single Responsibility: Each class/function should have one reason to change

Open/Closed: Open for extension, closed for modification

Liskov Substitution: Derived classes must be substitutable for their base classes

Interface Segregation: Clients should not depend on interfaces they don't use

Dependency Inversion: Depend on abstractions, not concretions

Reference: <https://en.wikipedia.org/wiki/SOLID>

DRY (Don't Repeat Yourself): Code reuse

Principle: Every piece of knowledge should have a single, unambiguous representation

Reduces duplication, makes maintenance easier

Reference: https://en.wikipedia.org/wiki/Don%27t_repeat_yourself

KISS (Keep It Simple, Stupid): Simplicity

Principle: Simplicity should be a key goal in design

Simple solutions are easier to understand, maintain, and debug

Reference: https://en.wikipedia.org/wiki/KISS_principle

YAGNI (You Aren't Gonna Need It): Avoid over-engineering

Principle: Don't add functionality until it's necessary

Prevents unnecessary complexity and technical debt

Reference: https://en.wikipedia.org/wiki/You_aren%27t_gonna_need_it

Documentation:

Markdown Guide: markdownguide.org

Comprehensive guide to Markdown syntax

Used for documentation in MyMasjidApp (README.md, DEVELOPMENT_STANDARDS.md, etc.)

Reference: <https://www.markdownguide.org/>

JSDoc: jsdoc.app

Documentation generator for JavaScript

Used for documenting JavaScript functions and classes

Reference: <https://jsdoc.app/>

React Documentation: react.dev

Official React documentation and guides

Covers hooks, components, state management, and best practices

Reference: <https://react.dev/>

Node.js Best Practices: github.com/goldbergonyi/nodebestpractices

Comprehensive collection of Node.js best practices

Covers project structure, error handling, security, performance, and testing

Reference: <https://github.com/goldbergonyi/nodebestpractices>

-

Version History:

Version 1.0 - January 2025 - Initial version

Created comprehensive development standards manual

Included coding standards, version control, architecture, testing, deployment, security, maintenance, tools, roles, compliance

Added project-specific examples and explanations

Enhanced with detailed explanations and reasoning

Review Schedule:

Quarterly Reviews: Review and update the manual every 3 months

After Major Changes: Update after significant technology changes, architecture changes, or process changes

On-Demand Updates: Update as standards evolve based on team feedback or new best practices

Team Feedback: Collect feedback from developers, QA, and project managers

Continuous Improvement: Regularly refine standards based on lessons learned

Update Process:

Identify areas that need updating (based on team feedback, technology changes, or new requirements)

Draft proposed changes (document the rationale for changes)

Review with team (discuss in team meetings or via pull request)

Get approval (consensus from team or approval from technical lead)

Update document (make changes and update version number)

Communicate changes (notify team of updates)

Contact:

Questions or Suggestions: Contact the development team or technical lead

Propose Changes: Create a pull request with proposed changes

Discuss Improvements: Bring up in team meetings or create an issue

Clarifications: Ask in team chat or create a discussion thread

Distribution:

This manual is available to all developers working on MyMasjidApp

New team members should read this manual as part of onboarding

Manual should be easily accessible (version control, shared drive, wiki)

Latest version should always be clearly marked

-

End of Development Standards Manual

Last Updated: January 2025 Version: 1.0 Maintained By: MyMasjidApp Development Team

API Documentation

Version: 1.0 Date: January 12, 2025 Base URL: <http://localhost:5000/api> (Development) Base URL: <https://yourdomain.com/api> (Production) ---

- [Authentication Endpoints](#authentication-endpoints)
- [Student Management Endpoints](#student-management-endpoints)
- [Teacher Management Endpoints](#teacher-management-endpoints)
- [Class Management Endpoints](#class-management-endpoints)
- [Attendance Endpoints](#attendance-endpoints)
- [Exam Management Endpoints](#exam-management-endpoints)
- [Results Management Endpoints](#results-management-endpoints)
- [Fee Management Endpoints](#fee-management-endpoints)
- [Admin Management Endpoints](#admin-management-endpoints)
- [Settings Endpoints](#settings-endpoints)
- [Announcements Endpoints](#announcements-endpoints)
- [Other Endpoints](#other-endpoints)
-

Description: Registers a new student user. Creates a user account with student role. Password is optional for student registration.

Authentication: Not required

```
{
  "nama": "Ahmad Zulkifli",
  "ic_number": "051003060229",
  "email": "ahmad@example.com",
  "password": "123456",
  "confirmPassword": "123456",
  "telepon": "0123456789",
  "umur": 20
}
```

Request Body:

Request Body Schema:

nama (string, required) - User full name (2-100 characters)

ic_number (string, required) - IC number (12 digits, format: XXXXXX-XX-XXXX or without hyphens)

email (string, optional) - Email address (must be valid format if provided)

password (string, optional) - Password (minimum 6 characters if provided)

confirmPassword (string, optional) - Password confirmation (must match password)

telefon (string, optional) - Phone number (1-20 characters)

umur (integer, optional) - Age (1-150)

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Registration successful",
  "data": {
    "ic": "051003060229",
    "nama": "Ahmad Zulkifli",
    "email": "ahmad@example.com",
    "role": "student"
  }
}
```

Response Body:

Error Responses:

Status Code: 400 Bad Request

```
{
  "success": false,
  "message": "Format IC tidak sah. Sila masukkan 12 digit nombor IC.",
  "errors": []
}
```

Response Body:

Status Code: 400 Bad Request (Duplicate IC)

```
{
  "success": false,
  "message": "Nombor IC ini sudah didaftarkan sebagai pelajar. Sila log masuk atau hubungi pentadbir jika anda memerlukan bantuan.",
}
```

```
"accountStatus": "already_registered"
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "nama": "Ahmad Zulkifli",
  "ic_number": "051003060229",
  "email": "ahmad@example.com",
  "password": "123456",
  "confirmPassword": "123456",
  "telepon": "0123456789",
  "umur": 20
}'
```

cURL:

```
const response = await axios.post('/api/auth/register', {
  nama: 'Ahmad Zulkifli',
  ic_number: '051003060229',
  email: 'ahmad@example.com',
  password: '123456',
  confirmPassword: '123456',
  telepon: '0123456789',
  umur: 20
});
```

JavaScript (Axios):

Notes:

IC number is normalized (hyphens removed) before storage

Password is optional for student registration

Email must be unique if provided

Rate limiting: 3 registrations per hour per IP address

Student role is automatically assigned

Related Endpoints:

POST /api/auth/login - Log in after registration

GET /api/auth/profile - Get user profile information

PUT /api/auth/profile - Update user profile

-

Description: Authenticates a user with IC number and password. Returns a JWT token and user information upon successful authentication.

Authentication: Not required

```
{
  "icNumber": "051003060229",
  "password": "123456"
}
```

Request Body:

Request Body Schema:

icNumber (string, required) - User IC number (12 digits, with or without hyphens)

password (string, required) - User password

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "ic": "051003060229",
      "nama": "Ahmad Zulkifli",
      "email": "ahmad@example.com",
      "role": "student",
      "roles": ["student"],
      "activeRole": "student"
    }
  }
}
```

Response Body:

Error Responses:

Status Code: 401 Unauthorized

```
{  
  "success": false,  
  "message": "Invalid credentials"  
}
```

Response Body:

Status Code: 429 Too Many Requests

```
{  
  "success": false,  
  "message": "Too many authentication attempts from this IP, please try again after 15 minutes."  
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/auth/login \  
-H "Content-Type: application/json" \  
-d '{  
  "icNumber": "051003060229",  
  "password": "123456"  
}'
```

cURL:

```
const response = await axios.post('/api/auth/login', {  
  icNumber: '051003060229',  
  password: '123456'  
});
```

JavaScript (Axios):

Notes:

Token expires after 24 hours

Rate limiting: 5 login attempts per 15 minutes per IP address

Account lockout after 5 failed attempts (30-minute lockout period)

IC number is normalized before authentication

Password is hashed using bcrypt (12 rounds)

Related Endpoints:

POST /api/auth/register - Register new user account

GET /api/auth/profile - Get authenticated user profile

POST /api/auth/forgot-password - Request password reset

PUT /api/auth/change-password - Change user password

-

Description: Retrieves the authenticated user's profile information.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": {
    "ic": "051003060229",
    "nama": "Ahmad Zulkifli",
    "email": "ahmad@example.com",
    "telepon": "0123456789",
    "umur": 20,
    "role": "student",
    "roles": ["student"],
    "activeRole": "student",
    "status": "aktif"
  }
}
```

Response Body:

Error Responses:

Status Code: 401 Unauthorized

```
{
  "success": false,
  "message": "Token not provided or invalid"
}
```

Response Body:

Example Request:

```
curl -X GET http://localhost:5000/api/auth/profile \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

cURL:

```
const response = await axios.get('/api/auth/profile', {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Requires valid JWT token

Returns user information based on authenticated user's IC number

Related Endpoints:

POST /api/auth/login - Authenticate to get token

PUT /api/auth/profile - Update user profile information

PUT /api/auth/change-password - Change user password

-

Description: Retrieves a list of students with optional filtering and pagination. Teachers can only see students in their assigned classes.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Query Parameters:

page (integer, optional) - Page number for pagination (default: 1)

limit (integer, optional) - Number of items per page (default: 1000, max: 1000)

search (string, optional) - Search term (searches in name and IC number)

kelas_id (integer, optional) - Filter by class ID

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": [
    {
      "ic": "051003060229",
      "nama": "Ahmad Zulkifli",
      "umur": 20,
      "email": "ahmad@example.com",
      "telepon": "0123456789",
      "role": "student",
      "status": "aktif",
      "kelas_id": 1,
      "nama_kelas": "Al-Quran Asas",
      "tarikh_daftar": "2025-01-15"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 1000,
    "total": 50,
    "totalPages": 1
  }
}
```

Response Body:

Error Responses:

Status Code: 401 Unauthorized

```
{
  "success": false,
  "message": "Token not provided or invalid"
}
```

Response Body:

Example Request:

```
curl -X GET "http://localhost:5000/api/students?page=1&limit=50&search=Ahmad" \
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.get('/api/students', {
  params: {
    page: 1,
    limit: 50,
    search: 'Ahmad'
  },
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Teachers can only see students in their assigned classes

Admins can see all students

Search works on name and IC number

Results are paginated

Related Endpoints:

GET /api/students/:ic - Get detailed student information

POST /api/students - Create new student record

GET /api/classes - Get list of classes for student assignment

GET /api/attendance?student_ic={ic} - Get student attendance records

-

Description: Retrieves detailed information about a specific student by IC number.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Path Parameters:

ic (string, required) - Student IC number (12 digits, with or without hyphens)

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": {
    "ic": "051003060229",
    "nama": "Ahmad Zulkifli",
    "umur": 20,
    "email": "ahmad@example.com",
    "telepon": "0123456789",
    "alamat": "Kampung Baru, Kuala Lumpur",
    "role": "student",
    "status": "aktif",
    "kelas_id": 1,
    "nama_kelas": "Al-Quran Asas",
    "tarikh_daftar": "2025-01-15",
    "created_at": "2025-01-15T10:00:00.000Z",
    "updated_at": "2025-01-15T10:00:00.000Z"
  }
}
```

Response Body:

Error Responses:

Status Code: 404 Not Found

```
{
  "success": false,
  "message": "Student not found"
}
```

Response Body:

Example Request:

```
curl -X GET http://localhost:5000/api/students/051003060229 \
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.get('/api/students/051003060229', {
  headers: {
    'Authorization': `Bearer ${token}`
  }
})
```

```
}  
});
```

JavaScript (Axios):

Notes:

IC number is normalized before query

Returns detailed student information including class details

Teachers can only access students in their classes

Related Endpoints:

GET /api/students - Get list of all students

PUT /api/students/:ic - Update student information

GET /api/attendance?student_ic={ic} - Get student attendance history

GET /api/results?student_ic={ic} - Get student exam results

GET /api/fees?student_ic={ic} - Get student fee records

-

Description: Creates a new student record. Only admins can create students.

Authentication: Required (Bearer token)

Authorization: Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

```
{  
  "nama": "Ahmad Zulkifli",  
  "ic": "051003060229",  
  "umur": 20,  
  "email": "ahmad@example.com",  
  "telepon": "0123456789",  
  "alamat": "Kampung Baru, Kuala Lumpur",  
  "kelas_id": 1  
}
```

Request Body:

Request Body Schema:

nama (string, required) - Student name (2-100 characters)

ic (string, required) - IC number (12 digits, format: XXXXXX-XX-XXXX)

umur (integer, optional) - Age (5-100)

email (string, optional) - Email address (valid format)

telepon (string, optional) - Phone number (valid format)

alamat (string, optional) - Address (10-500 characters)

kelas_id (integer, optional) - Class ID

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Student created successfully",
  "data": {
    "ic": "051003060229",
    "nama": "Ahmad Zulkifli",
    "umur": 20,
    "email": "ahmad@example.com",
    "telepon": "0123456789",
    "role": "student",
    "status": "aktif"
  }
}
```

Response Body:

Error Responses:

Status Code: 400 Bad Request (Validation Error)

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "field": "nama",
      "message": "Name must be between 2 and 100 characters"
    }
  ]
}
```

```
]
}
```

Response Body:

Status Code: 403 Forbidden

```
{
  "success": false,
  "message": "Insufficient permissions. Admin role required."
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/students \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "nama": "Ahmad Zulkifli",
  "ic": "051003060229",
  "umur": 20,
  "email": "ahmad@example.com",
  "telepon": "0123456789",
  "alamat": "Kampung Baru, Kuala Lumpur",
  "kelas_id": 1
}'
```

cURL:

```
const response = await axios.post('/api/students', {
  nama: 'Ahmad Zulkifli',
  ic: '051003060229',
  umur: 20,
  email: 'ahmad@example.com',
  telepon: '0123456789',
  alamat: 'Kampung Baru, Kuala Lumpur',
  kelas_id: 1
}, {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Only admins can create students

IC number must be unique

IC number is normalized before storage

Email must be unique if provided

Creates user record and student record

Related Endpoints:

GET /api/students/:ic - Get created student details

GET /api/students - List all students

PUT /api/students/:ic - Update student information

GET /api/classes - Get available classes for assignment

-

Description: Updates an existing student record. Only admins can update students.

Authentication: Required (Bearer token)

Authorization: Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Path Parameters:

ic (string, required) - Student IC number (12 digits, with or without hyphens)

```
{
  "nama": "Ahmad Zulkifli Updated",
  "umur": 21,
  "email": "ahmad.updated@example.com",
  "telepon": "0123456789",
  "alamat": "New Address",
  "kelas_id": 2
}
```

Request Body:

Request Body Schema:

nama (string, optional) - Student name (2-100 characters)

umur (integer, optional) - Age (5-100)

email (string, optional) - Email address (valid format)

telepon (string, optional) - Phone number (valid format)

alamat (string, optional) - Address (10-500 characters)

kelas_id (integer, optional) - Class ID

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "message": "Student updated successfully",
  "data": {
    "ic": "051003060229",
    "nama": "Ahmad Zulkifli Updated",
    "umur": 21,
    "email": "ahmad.updated@example.com"
  }
}
```

Response Body:

Error Responses:

Status Code: 404 Not Found

```
{
  "success": false,
  "message": "Student not found"
}
```

Response Body:

Example Request:

```
curl -X PUT http://localhost:5000/api/students/051003060229 \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "nama": "Ahmad Zulkifli Updated",
```

```
"umur": 21,  
"email": "ahmad.updated@example.com"  
'
```

cURL:

```
const response = await axios.put('/api/students/051003060229', {  
  nama: 'Ahmad Zulkifli Updated',  
  umur: 21,  
  email: 'ahmad.updated@example.com'  
}, {  
  headers: {  
    'Authorization': `Bearer ${token}`  
  }  
});
```

JavaScript (Axios):

Notes:

Only admins can update students

IC number cannot be changed

Email must be unique if provided

Only provided fields are updated

Related Endpoints:

GET /api/students/:ic - Get student details before updating

GET /api/students - List all students

POST /api/students - Create new student

DELETE /api/students/:ic - Delete student record

-

Description: Deletes a student record. Only admins can delete students.

Authentication: Required (Bearer token)

Authorization: Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Path Parameters:

ic (string, required) - Student IC number (12 digits, with or without hyphens)

Success Response:

Status Code: 200 OK

```
{  
  "success": true,  
  "message": "Student deleted successfully"  
}
```

Response Body:

Error Responses:

Status Code: 404 Not Found

```
{  
  "success": false,  
  "message": "Student not found"  
}
```

Response Body:

Example Request:

```
curl -X DELETE http://localhost:5000/api/students/051003060229 \  
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.delete('/api/students/051003060229', {  
  headers: {  
    'Authorization': `Bearer ${token}`  
  }  
});
```

JavaScript (Axios):

Notes:

Only admins can delete students

Deletion is cascaded (related records in attendance, results, fees are also deleted)

IC number is normalized before deletion

Action is logged for audit purposes

Related Endpoints:

GET /api/students/:ic - Get student details before deletion

GET /api/students - List all students

POST /api/students - Create new student

GET /api/admin-actions - View admin action history (including deletions)

-

Description: Retrieves a list of teachers with optional filtering and pagination.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Query Parameters:

page (integer, optional) - Page number for pagination (default: 1)

limit (integer, optional) - Number of items per page (default: 50, max: 1000)

search (string, optional) - Search term (searches in name and IC number)

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": [
    {
      "ic": "820503060229",
      "nama": "Ustaz Rahim",
      "umur": 42,
      "email": "rahim@example.com",
      "telepon": "0172233445",
      "role": "teacher",
      "status": "aktif",
      "kepakaran": ["Al-Quran", "Tajwid"]
    }
  ],
}
```

```
"pagination": {  
  "page": 1,  
  "limit": 50,  
  "total": 10,  
  "totalPages": 1  
}
```

Response Body:

Error Responses:

Status Code: 401 Unauthorized

```
{  
  "success": false,  
  "message": "Token not provided or invalid"  
}
```

Response Body:

Example Request:

```
curl -X GET "http://localhost:5000/api/teachers?page=1&limit=50" \  
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.get('/api/teachers', {  
  params: {  
    page: 1,  
    limit: 50  
  },  
  headers: {  
    'Authorization': `Bearer ${token}`  
  }  
});
```

JavaScript (Axios):

Notes:

Returns list of all teachers

Search works on name and IC number

Results are paginated

Kepakaran (expertise) is returned as JSON array

Related Endpoints:

GET /api/teachers/:ic - Get detailed teacher information

POST /api/teachers - Create new teacher record

GET /api/classes?guru_ic={ic} - Get classes assigned to teacher

-

Description: Creates a new teacher record. Only admins can create teachers.

Authentication: Required (Bearer token)

Authorization: Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

```
{
  "nama": "Ustaz Rahim",
  "ic": "820503060229",
  "umur": 42,
  "email": "rahim@example.com",
  "telepon": "0172233445",
  "alamat": "Bangi, Selangor",
  "kepakaran": ["Al-Quran", "Tajwid"]
}
```

Request Body:

Request Body Schema:

nama (string, required) - Teacher name (2-100 characters)

ic (string, required) - IC number (12 digits)

umur (integer, optional) - Age (5-100)

email (string, optional) - Email address (valid format)

telepon (string, optional) - Phone number (valid format)

alamat (string, optional) - Address (10-500 characters)

kepakaran (array, optional) - Array of expertise/specializations

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Teacher created successfully",
  "data": {
    "ic": "820503060229",
    "nama": "Ustaz Rahim",
    "role": "teacher",
    "status": "aktif",
    "kepakaran": ["Al-Quran", "Tajwid"]
  }
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/teachers \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "nama": "Ustaz Rahim",
  "ic": "820503060229",
  "umur": 42,
  "email": "rahim@example.com",
  "telepon": "0172233445",
  "kepakaran": ["Al-Quran", "Tajwid"]
}'
```

cURL:

```
const response = await axios.post('/api/teachers', {
  nama: 'Ustaz Rahim',
  ic: '820503060229',
  umur: 42,
  email: 'rahim@example.com',
  telepon: '0172233445',
  kepakaran: ['Al-Quran', 'Tajwid']
}, {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```


JavaScript (Axios):

Notes:

Only admins can create teachers

IC number must be unique

Kepakaran is stored as JSON array

Creates user record and teacher record

Related Endpoints:

GET /api/teachers - List all teachers

GET /api/teachers/:ic - Get created teacher details

POST /api/classes - Assign teacher to class

GET /api/classes?guru_ic={ic} - View teacher's assigned classes

-

Description: Retrieves a list of classes with optional filtering and pagination.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Query Parameters:

page (integer, optional) - Page number for pagination (default: 1)

limit (integer, optional) - Number of items per page (default: 50, max: 1000)

search (string, optional) - Search term (searches in class name)

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": [
    {
      "id": 1,
```

```

    "nama_kelas": "Al-Quran Asas",
    "level": "Asas",
    "jadual": "Isnin & Rabu 5:00AM-6:30AM",
    "sessions": ["Isnin", "Rabu"],
    "yuran": 150.00,
    "guru_ic": "820503060229",
    "guru_nama": "Ustaz Rahim",
    "kapasiti": 20,
    "status": "aktif",
    "created_at": "2025-01-01T00:00:00.000Z"
  }
],
"pagination": {
  "page": 1,
  "limit": 50,
  "total": 5,
  "totalPages": 1
}
}

```

Response Body:

Example Request:

```

curl -X GET "http://localhost:5000/api/classes?page=1&limit=50" \
-H "Authorization: Bearer {token}"

```

cURL:

```

const response = await axios.get('/api/classes', {
  params: {
    page: 1,
    limit: 50
  },
  headers: {
    'Authorization': `Bearer ${token}`
  }
});

```

JavaScript (Axios):

Notes:

Sessions is stored as JSON array

Returns teacher information if assigned

Results are paginated

Related Endpoints:

GET /api/classes/:id - Get detailed class information

POST /api/classes - Create new class

GET /api/students?kelas_id={id} - Get students in class

GET /api/attendance?class_id={id} - Get class attendance records

-

Description: Creates a new class. Only admins can create classes.

Authentication: Required (Bearer token)

Authorization: Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

```
{
  "nama_kelas": "Al-Quran Asas",
  "level": "Asas",
  "jadual": "Isnin & Rabu 5:00AM-6:30AM",
  "sessions": ["Isnin", "Rabu"],
  "yuran": 150.00,
  "guru_ic": "820503060229",
  "kapasiti": 20,
  "status": "aktif"
}
```

Request Body:

Request Body Schema:

nama_kelas (string, required) - Class name (1-100 characters)

level (string, optional) - Class level

jadual (string, optional) - Schedule description

sessions (array, optional) - Array of session days (e.g., ["Isnin", "Rabu"])

yuran (decimal, optional) - Monthly fee amount (default: 0)

guru_ic (string, optional) - Teacher IC number

kapasiti (integer, optional) - Class capacity (default: 20)

status (string, optional) - Class status: "aktif", "tidak_aktif", "penuh" (default: "aktif")

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Class created successfully",
  "data": {
    "id": 1,
    "nama_kelas": "Al-Quran Asas",
    "level": "Asas",
    "yuran": 150.00,
    "kapasiti": 20,
    "status": "aktif"
  }
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/classes \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "nama_kelas": "Al-Quran Asas",
  "level": "Asas",
  "jadual": "Isnin & Rabu 5:00AM-6:30AM",
  "sessions": ["Isnin", "Rabu"],
  "yuran": 150.00,
  "guru_ic": "820503060229",
  "kapasiti": 20
}'
```

cURL:

```
const response = await axios.post('/api/classes', {
  nama_kelas: 'Al-Quran Asas',
  level: 'Asas',
  jadual: 'Isnin & Rabu 5:00AM-6:30AM',
  sessions: ['Isnin', 'Rabu'],
```

```
yuran: 150.00,  
guru_ic: '820503060229',  
kapasiti: 20  
}, {  
  headers: {  
    'Authorization': `Bearer ${token}`  
  }  
});
```

JavaScript (Axios):

Notes:

Only admins can create classes

Sessions must be valid JSON array

Teacher IC must exist if provided

Yuran (fee) is stored as DECIMAL(10,2)

Related Endpoints:

GET /api/classes - List all classes

GET /api/classes/:id - Get created class details

GET /api/teachers - Get available teachers for assignment

POST /api/students - Assign students to class

-

Description: Retrieves attendance records with optional filtering by student, class, and date range.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Query Parameters:

page (integer, optional) - Page number for pagination (default: 1)

limit (integer, optional) - Number of items per page (default: 50, max: 1000)

student_ic (string, optional) - Filter by student IC number

class_id (integer, optional) - Filter by class ID

start_date (date, optional) - Start date (format: YYYY-MM-DD)

end_date (date, optional) - End date (format: YYYY-MM-DD)

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "student_ic": "051003060229",
      "student_nama": "Ahmad Zulkifli",
      "class_id": 1,
      "class_nama": "Al-Quran Asas",
      "tarikh": "2025-01-15",
      "status": "Hadir",
      "catatan": null,
      "created_at": "2025-01-15T10:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 50,
    "total": 100,
    "totalPages": 2
  }
}
```

Response Body:

Example Request:

```
curl -X GET "http://localhost:5000/api/attendance?class_id=1&start_date=2025-01-01&end_date=2025-01-31" \
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.get('/api/attendance', {
  params: {
    class_id: 1,
    start_date: '2025-01-01',
```

```
    end_date: '2025-01-31'
  },
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Teachers can only see attendance for their classes

Admins can see all attendance records

Date filtering is inclusive

Status values: "Hadir", "Tidak Hadir", "Cuti"

Related Endpoints:

POST /api/attendance - Create new attendance record

GET /api/attendance/student/:student_ic - Get student attendance history

GET /api/students/:ic - Get student details

GET /api/classes/:id - Get class information

-

Description: Creates a new attendance record. Teachers and admins can create attendance records.

Authentication: Required (Bearer token)

Authorization: Teacher or Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

```
{
  "student_ic": "051003060229",
  "class_id": 1,
  "tarikh": "2025-01-15",
  "status": "Hadir",
  "catatan": "Hadir tepat pada masanya"
}
```

Request Body:

Request Body Schema:

student_ic (string, required) - Student IC number (12 digits)

class_id (integer, required) - Class ID

tarikh (date, required) - Attendance date (format: YYYY-MM-DD)

status (string, required) - Attendance status: "Hadir", "Tidak Hadir", "Cuti"

catatan (string, optional) - Notes/comments

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Attendance recorded successfully",
  "data": {
    "id": 1,
    "student_ic": "051003060229",
    "class_id": 1,
    "tarikh": "2025-01-15",
    "status": "Hadir",
    "catatan": "Hadir tepat pada masanya"
  }
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/attendance \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "student_ic": "051003060229",
  "class_id": 1,
  "tarikh": "2025-01-15",
  "status": "Hadir",
  "catatan": "Hadir tepat pada masanya"
}'
```

cURL:


```
const response = await axios.post('/api/attendance', {
  student_ic: '051003060229',
  class_id: 1,
  tarikh: '2025-01-15',
  status: 'Hadir',
  catatan: 'Hadir tepat pada masanya'
}, {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Teachers can only create attendance for their classes

Admins can create attendance for any class

Date format must be YYYY-MM-DD

Duplicate attendance for same student/class/date will update existing record

Related Endpoints:

GET /api/attendance - Get attendance records

GET /api/students/:ic - Verify student exists

GET /api/classes/:id - Verify class exists

PUT /api/attendance/:id - Update existing attendance record

-

Description: Retrieves a list of exams with optional filtering by class.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Query Parameters:

page (integer, optional) - Page number for pagination (default: 1)

limit (integer, optional) - Number of items per page (default: 50, max: 1000)

class_id (integer, optional) - Filter by class ID

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "class_id": 1,
      "class_nama": "Al-Quran Asas",
      "subject": "Tilawah Al-Quran",
      "tarikh_exam": "2025-09-15",
      "created_at": "2025-01-01T00:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 50,
    "total": 10,
    "totalPages": 1
  }
}
```

Response Body:

Example Request:

```
curl -X GET "http://localhost:5000/api/exams?class_id=1" \
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.get('/api/exams', {
  params: {
    class_id: 1
  },
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Returns exams with class information

Results are paginated

Can filter by class ID

Related Endpoints:

POST /api/exams - Create new exam

GET /api/exams/:id - Get detailed exam information

GET /api/results?exam_id={id} - Get results for exam

GET /api/classes/:id - Get class information

-

Description: Creates a new exam. Only admins can create exams.

Authentication: Required (Bearer token)

Authorization: Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

```
{
  "class_id": 1,
  "subject": "Tilawah Al-Quran",
  "tarikh_exam": "2025-09-15"
}
```

Request Body:

Request Body Schema:

class_id (integer, required) - Class ID

subject (string, required) - Exam subject/title

tarikh_exam (date, required) - Exam date (format: YYYY-MM-DD, ISO 8601)

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Exam created successfully",
  "data": {
    "id": 1,
    "class_id": 1,
    "subject": "Tilawah Al-Quran",
    "tarikh_exam": "2025-09-15"
  }
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/exams \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "class_id": 1,
  "subject": "Tilawah Al-Quran",
  "tarikh_exam": "2025-09-15"
}'
```

cURL:

```
const response = await axios.post('/api/exams', {
  class_id: 1,
  subject: 'Tilawah Al-Quran',
  tarikh_exam: '2025-09-15'
}, {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Only admins can create exams

Class ID must exist

Date format must be ISO 8601 (YYYY-MM-DD)

Related Endpoints:

GET /api/exams - List all exams

GET /api/exams/:id - Get created exam details

GET /api/classes/:id - Verify class exists

POST /api/results - Create results for exam

-

Description: Retrieves exam results with optional filtering by student, exam, and grade.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Query Parameters:

page (integer, optional) - Page number for pagination (default: 1)

limit (integer, optional) - Number of items per page (default: 50, max: 1000)

student_ic (string, optional) - Filter by student IC number

exam_id (integer, optional) - Filter by exam ID

gred (string, optional) - Filter by grade (A+, A, A-, B+, B, B-, C+, C, C-, D, F)

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "student_ic": "051003060229",
      "student_nama": "Ahmad Zulkifli",
      "exam_id": 1,
      "exam_subject": "Tilawah Al-Quran",
      "markah": 88,
      "gred": "A",
      "status": "lulus",
      "slip_img": "uploads/slip_ahmad.png",
      "catatan": "Prestasi cemerlang",
    }
  ]
}
```

```
    "created_at": "2025-09-20T10:00:00.000Z"
  }
],
"pagination": {
  "page": 1,
  "limit": 50,
  "total": 50,
  "totalPages": 1
}
}
```

Response Body:

Example Request:

```
curl -X GET "http://localhost:5000/api/results?student_ic=051003060229&exam_id=1" \
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.get('/api/results', {
  params: {
    student_ic: '051003060229',
    exam_id: 1
  },
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Students can only see their own results

Teachers can see results for their classes

Admins can see all results

Grade is automatically calculated based on markah (marks) and grade ranges

Status is automatically determined: "lulus" (pass) or "gagal" (fail)

Related Endpoints:

POST /api/results - Create new exam result

GET /api/results/:id - Get detailed result information

GET /api/exams/:id - Get exam details

GET /api/students/:ic - Get student information

-

Description: Creates a new exam result. Teachers and admins can create results.

Authentication: Required (Bearer token)

Authorization: Teacher or Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

```
{
  "student_ic": "051003060229",
  "exam_id": 1,
  "markah": 88,
  "gred": "A",
  "slip_img": "uploads/slip_ahmad.png",
  "catatan": "Prestasi cemerlang"
}
```

Request Body:

Request Body Schema:

student_ic (string, required) - Student IC number (format: XXXXXX-XX-XXXX)

exam_id (integer, required) - Exam ID

markah (integer, required) - Marks (0-100)

gred (string, optional) - Grade: A+, A, A-, B+, B, B-, C+, C, C-, D, F (auto-calculated if not provided)

slip_img (string, optional) - Result slip image path

catatan (string, optional) - Notes/comments

status (string, optional) - Status: "lulus" or "gagal" (auto-determined if not provided)

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Result created successfully",
  "data": {
    "id": 1,
    "student_ic": "051003060229",
    "exam_id": 1,
    "markah": 88,
    "gred": "A",
    "status": "lulus"
  }
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/results \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "student_ic": "051003060229",
  "exam_id": 1,
  "markah": 88,
  "gred": "A",
  "catatan": "Prestasi cemerlang"
}'
```

cURL:

```
const response = await axios.post('/api/results', {
  student_ic: '051003060229',
  exam_id: 1,
  markah: 88,
  gred: 'A',
  catatan: 'Prestasi cemerlang'
}, {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Grade can be auto-calculated based on markah if not provided

Status is auto-determined: "lulus" (pass) if grade is A+, A, A-, B+, B, B-, C+, C, C- or "gagal" (fail) if grade is D or F

Teachers can only create results for exams in their classes

Admins can create results for any exam

Related Endpoints:

GET /api/results - List all results

GET /api/results/:id - Get created result details

GET /api/exams/:id - Verify exam exists

GET /api/students/:ic - Verify student exists

GET /api/settings/grade-ranges - Get grade range configuration

-

Description: Retrieves fee records with optional filtering by student, status, month, and year.

Authentication: Required (Bearer token)

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

Query Parameters:

page (integer, optional) - Page number for pagination (default: 1)

limit (integer, optional) - Number of items per page (default: 50, max: 1000)

student_ic (string, optional) - Filter by student IC number

status (string, optional) - Filter by status: "Bayar", "Belum Bayar", "terbayar", "tunggak", "pending"

bulan (string, optional) - Filter by month (e.g., "Januari", "Februari")

tahun (integer, optional) - Filter by year

Success Response:

Status Code: 200 OK

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "student_ic": "051003060229",
      "student_nama": "Ahmad Zulkifli",
      "jumlah": 150.00,
      "status": "terbayar",
      "tarikh": "2025-02-01",
      "tarikh_bayar": "2025-02-01",
      "bulan": "Februari",
      "tahun": 2025,
      "cara_bayar": "Tunai",
      "no_resit": "R001",
      "resit_img": "uploads/resit_ahmad.png",
      "created_at": "2025-02-01T10:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 50,
    "total": 100,
    "totalPages": 2
  }
}
```

Response Body:

Example Request:

```
curl -X GET "http://localhost:5000/api/fees?student_ic=051003060229&status=tunggak" \
-H "Authorization: Bearer {token}"
```

cURL:

```
const response = await axios.get('/api/fees', {
  params: {
    student_ic: '051003060229',
    status: 'tunggak'
  },
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Students can only see their own fees

Admins and PIC users can see all fees

Status values: "Bayar", "Belum Bayar", "terbayar", "tunggak", "pending"

Results are paginated

Related Endpoints:

POST /api/fees - Create new fee record

GET /api/fees/:id - Get detailed fee information

PUT /api/fees/:id/mark-paid - Mark fee as paid

GET /api/students/:ic - Get student information

-

Description: Creates a new fee record. Only admins can create fees.

Authentication: Required (Bearer token)

Authorization: Admin role required

Authorization: Bearer {token}

Content-Type: application/json

Required Headers:

```
{
  "student_ic": "051003060229",
  "jumlah": 150.00,
  "tarikh": "2025-02-01",
  "bulan": "Februari",
  "tahun": 2025,
  "status": "Belum Bayar"
}
```

Request Body:

Request Body Schema:

student_ic (string, required) - Student IC number (12 digits)

jumlah (decimal, required) - Fee amount

tarikh (date, required) - Fee date (format: YYYY-MM-DD)

bulan (string, optional) - Month name (e.g., "Januari", "Februari")

tahun (integer, optional) - Year

status (string, optional) - Fee status: "Bayar", "Belum Bayar", "terbayar", "tunggak", "pending"
(default: "Belum Bayar")

Success Response:

Status Code: 201 Created

```
{
  "success": true,
  "message": "Fee created successfully",
  "data": {
    "id": 1,
    "student_ic": "051003060229",
    "jumlah": 150.00,
    "status": "Belum Bayar",
    "tarikh": "2025-02-01",
    "bulan": "Februari",
    "tahun": 2025
  }
}
```

Response Body:

Example Request:

```
curl -X POST http://localhost:5000/api/fees \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{
  "student_ic": "051003060229",
  "jumlah": 150.00,
  "tarikh": "2025-02-01",
  "bulan": "Februari",
  "tahun": 2025,
  "status": "Belum Bayar"
}'
```

cURL:

```
const response = await axios.post('/api/fees', {
  student_ic: '051003060229',
  jumlah: 150.00,
  tarikh: '2025-02-01',
  bulan: 'Februari',
  tahun: 2025,
  status: 'Belum Bayar'
}, {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

JavaScript (Axios):

Notes:

Only admins can create fees

Jumlah (amount) is stored as DECIMAL(10,2)

Status defaults to "Belum Bayar" if not provided

Fee can be marked as paid later with payment details

Related Endpoints:

GET /api/fees - List all fees

GET /api/fees/:id - Get created fee details

PUT /api/fees/:id/mark-paid - Mark fee as paid

GET /api/students/:ic - Verify student exists

POST /api/fees/generate-monthly - Generate monthly fees for all students

-

Authorization: Bearer {token}

All protected endpoints require a JWT token in the Authorization header:

Tokens are obtained via the /api/auth/login endpoint and expire after 24 hours.

Authentication endpoints: 5 requests per 15 minutes per IP

Registration endpoints: 3 requests per hour per IP

General API endpoints: 1000 requests per 15 minutes per IP

```
{
  "success": false,
  "message": "Error message",
  "errors": [
    {
      "field": "field-name",
      "message": "Field-specific error message"
    }
  ]
}
```

All error responses follow this format:

```
{
  "pagination": {
    "page": 1,
    "limit": 50,
    "total": 100,
    "totalPages": 2
  }
}
```

Paginated responses include a pagination object:

All dates should be in ISO 8601 format: YYYY-MM-DD (e.g., 2025-01-15)

IC numbers should be 12 digits. They can be provided with or without hyphens:

With hyphens: 051003-06-0229

Without hyphens: 051003060229

The system normalizes IC numbers (removes hyphens) before storage and queries.

```
{
  "success": true,
  "message": "Success message",
  "data": { ... }
}
```

All successful responses follow this format:

-

Document Version: 1.0 Last Updated: January 12, 2025 Maintained By: MyMasjidApp Development Team

Current System Information

Version Information

Frontend Version: 0.0.0

Backend Version: 1.0.0

React Version: 19.1.1

Node.js Version: LTS

MySQL Version: 8.0

Docker Services

- masjid_backend: Running
- masjid_frontend: Running
- masjid_mysql: Running
- masjid_nginx: Running

System Features

- Student Management
- Teacher Management
- Class Management
- Attendance Tracking
- Exam Management
- Results Management
- Fee Management
- Payment Gateway Integration (ToyyibPay)
- User Authentication & Authorization
- Role-Based Access Control (Student, Teacher, Admin, IB, PIC)
- Yearly Database System
- Announcements
- Notifications
- Staff Check-In
- Audit Logs
- System Health Monitoring
- Help Center
- Password Reset (Email/SMS)
- Profile Management
- Settings Management

API Endpoints

- /api/adminActions
- /api/admins
- /api/announcements
- /api/archive
- /api/attendance
- /api/auth
- /api/classes
- /api/contact
- /api/exams
- /api/export
- /api/fees
- /api/googleForm
- /api/ib
- /api/maintenance
- /api/migration
- /api/paymentGatewaySettings
- /api/payments
- /api/pendingPicChanges
- /api/picRecycleBin
- /api/picUsers
- /api/quranQuote
- /api/receipts
- /api/results
- /api/settings
- /api/staffCheckIn
- /api/students
- /api/teachers
- /api/toyyibpay
- /api/users
- /api/vllm
- /api/weather

Frontend Routes

- /forgot-password
- /contact
- /kehadiran
- /permission-matrix
- /quick-checkin
- /payment/return
- /settings
- /reset-password

- /notifications
- /pic-recycle-bin
- /choose-reset-method
- /
- *
- /staff-checkin
- /announcements
- /weather
- /keputusan
- /hierarchy
- /personal-settings
- /admin-actions
- /login
- /pelajar/*
- /account
- /all-users
- /admins
- /student-register
- /toyyibpay-settings
- /kelas/*
- /help
- /teacher-register

Last Updated: 2026-01-12 14:24:47



Current System Information



Version Information

✓ Frontend Version: 0.0.0

✓ Backend Version: 1.0.0

✓ React Version: 19.1.1

✓ Node.js Version: LTS

✓ MySQL Version: 8.0



Docker Services Status

🟢 masjid_backend: Running

🟢 masjid_frontend: Running

🟢 masjid_mysql: Running

🟢 masjid_nginx: Running



System Features

- Student Management
- Teacher Management
- Class Management
- Attendance Tracking
- Exam Management
- Results Management
- Fee Management
- Payment Gateway Integration (ToyyibPay)
- User Authentication & Authorization

- Role-Based Access Control (Student, Teacher, Admin, IB, PIC)
- Yearly Database System
- Announcements
- Notifications
- Staff Check-In
- Audit Logs
- System Health Monitoring
- Help Center
- Password Reset (Email/SMS)
- Profile Management
- Settings Management

API Endpoints

- /api/adminActions
- /api/admins
- /api/announcements
- /api/archive
- /api/attendance
- /api/auth
- /api/classes
- /api/contact
- /api/exams
- /api/export
- /api/fees
- /api/googleForm
- /api/ib
- /api/maintenance

- /api/migration
- /api/paymentGatewaySettings
- /api/payments
- /api/pendingPicChanges
- /api/picRecycleBin
- /api/picUsers
- ... and 11 more endpoints

Frontend Routes

- /payment-history
- /register
- /pelajar/*
- /reset-password-code
- /yuran
- /guru/*
- /kelas/*
- /pay-yuran/:id
- /quick-checkin
- /staff-checkin
- /payment/return
- /help
- /ib-account
- /laporan
- /account
- /permission-matrix
- /keputusan
- /system-health

→ /pic-recycle-bin

→ /admin-actions

→ /all-users

→ /personal-settings

→ /choose-reset-method

→ /

→ /pic-users

... and 23 more routes

🕒 Last Updated: 2026-01-12 14:32:33